第3章: Web基礎・HTML・CSS ― 画面の骨組みと見た目（完全版）

> この章では、Webページが表示されるしくみと、画面の **骨組みを作るHTML・見た目を整えるCSS** を、**本アプリの実際のスタイルファイルを1行残らず読み解きながら** 学びます。本アプリのフロント（前面）は React（リアクト）で書かれていますが、ブラウザが最終的に画面へ描き出すのは結局 **HTMLとCSS** です。ここを理解すれば、「画面のこの文字を変えたい」「この色を変えたい」「この余白を詰めたい」を、自分の手で安全にできるようになります。
>
> プログラミング完全初心者の方を想定しています。専門用語は出てくるたびに「用語（よみ：意味）」の形でかみ砕き、たとえ話を多用します。コードは **すべて本アプリの実物** を引用し、説明のために作った簡易例には「※説明用の簡易例」と明記します。
>
> この章は長いですが、**辞書のように使えること** を目指しています。実務で「あのプロパティ何だっけ」「この菱形ボタンどうやって作ってるんだっけ」と思ったら、ここに戻って該当箇所を引いてください。

この章で学ぶこと

- **ブラウザが画面を作るしくみ** ― HTTPでHTMLを受け取り、DOM（ドム）を組み立て、CSSを当てて画面へ描く（レンダリング）までの流れ
- **HTMLの全体** ― タグ・属性・入れ子・空要素・セマンティック（意味のある）タグ・よく使うタグ早見表
- **HTMLフォーム** ― `form` / `label` / `input`（全type）/ `button` / `select` / `textarea`。本アプリは入力フォームの塊
- **`Login.tsx` のJSXを1行ずつ完全解説** ― `class`→`className`、`for`→`htmlFor`、`{ }` での値の埋め込み
- **CSSの全体** ― セレクタ・カスケード（優先順位）・ボックスモデル・単位・色・Flexbox・擬似クラス/擬似要素・トランジション・アニメーション・メディアクエリ
- **本アプリの `global.css` を全342行、1行ずつ解説** ― リセット・CSS変数・中央寄せレイアウト・色違いボタン7種・表・アラート・日付入力の調整
- **本アプリの `Login.module.css` を全100行、1行ずつ解説** ― 出現アニメーション・入力フォーカス・菱形ヘルプボタン
- **`KosuList` / `MainMenu` / `Pagination` の実CSS** ― 一覧画面・メニュー画面・共通部品の実例
- **CSS Modules（シーエスエス・モジュールズ）** ― 名前衝突を防ぐしくみ
- **開発者ツール（F12）** ― 見た目調整・調査の実践手順
- **アクセシビリティ・よくある落とし穴・トラブルシューティング・演習問題**

目次

0. 前提知識：ブラウザが画面を作るしくみ（DOM・レンダリング）
1. HTMLの基本
2. HTMLフォーム（本アプリの心臓）
3. 実例：Login.tsx のJSXを1行ずつ完全解説
4. CSSの基本文法
5. ボックスモデル（余白の正体）
6. 単位と色
7. Flexbox（配置の主役）
8. 擬似クラスと擬似要素
9. トランジションとアニメーション
10. メディアクエリ（レスポンシブ）
11. 実コード全解説：global.css 全342行を1行ずつ
12. 実コード全解説：Login.module.css 全100行を1行ずつ
13. 実コード追加解説：KosuList / MainMenu / Pagination
14. CSS Modulesのしくみ
15. 開発者ツール（F12）実践
16. アクセシビリティ
17. よくある落とし穴とトラブルシューティング
18. 演習問題
19. この章のまとめ

---

0. 前提知識：ブラウザが画面を作るしくみ（DOM・レンダリング）

「この章のコードを書き換えると、なぜ画面が変わるのか」を理解するために、まず **ブラウザ（Chrome・Edge など）の内部** を簡単にのぞいておきます。ここが分かると、後のCSS解説が「ただの呪文」ではなく「ちゃんと意味のある指示」に見えてきます。

0.1 そもそも「Webページが見える」とは何が起きているのか

> **ブラウザ（browser：ブラウザ）とは？** Webページを見るためのソフト。Google Chrome（グーグル・クローム）、Microsoft Edge（マイクロソフト・エッジ）、Safari（サファリ）などがあります。本アプリも、従業員はブラウザで開いて使います。
>
> **サーバー（server：サーバー）とは？** ページのデータ（HTMLや画像）を保管し、ブラウザから「ください」と言われたら渡すコンピュータ。本アプリでは、開発中は `http://localhost:3000`（フロント）や `http://localhost:8000`（バックエンド）が「自分のパソコンの中のサーバー」です。

あなたが `http://localhost:3000` を開くと、ブラウザは次の手順を踏みます。

コピー

```
1. サーバーに「このURLのページをください」とお願いする（HTTPリクエスト）
2. サーバーから HTML文書（ただの長い文字列）を受け取る（HTTPレスポンス）
3. HTMLを解析して「DOMツリー」という木構造のデータに組み立てる
4. CSS（見た目の指示）を解析して「どの要素にどんな見た目を当てるか」を決める
5. DOM + CSS を合体させて「レンダリングツリー」を作り、画面にピクセルとして描く（レンダリング）
6. JavaScript が DOM を書き換えると、3〜5 がやり直され、画面が更新される
```

> **HTTP（エイチティーティーピー：HyperText Transfer Protocol）とは？** ブラウザとサーバーが会話するときの「言葉のルール（プロトコル）」。「このページください（GET）」「このデータ保存して（POST）」といったお願いの形式が決まっています。本アプリの通信もこのHTTPで行われます（詳しくは第4章・第7章）。
>
> **レンダリング（rendering：レンダリング）とは？**「文字や色や配置の指示」を、実際に目に見える絵（ピクセル）として画面に描き出すこと。日本語では「描画」とも言います。CSSを変えると見た目が変わるのは、この再レンダリングが起きるからです。

0.2 DOM（ドム）— HTMLが「木」になる

> **DOM（ドム：Document Object Model）とは？** HTMLを、プログラムから操作できる「木構造（ツリー）のオブジェクト」として表したもの。「Document（文書）」を「Object（もの）」の「Model（模型）」にしたもの、という意味です。

たとえば次のHTMLは…

コピー

```
<div>
  <h1>タイトル</h1>
  <p>本文</p>
</div>
```

▶ ここで初めて出てくるコード

- `<div>`（HTML）：特別な意味を持たない「箱」のタグ。CSS で見た目や並べ方を決めるために、ほかのタグをまとめる時に使います。
- `<h1>`（HTML）：いちばん大きな見出し（heading 1）のタグ。数が大きいほど小さな見出しになります（`<h2>` 〜 `<h6>`）。`<h1>` は、1 ページに 1 つにします。
- `<p>`（HTML）：段落（paragraph）のタグ。ひとまとまりの文を囲みます。段落と段落の間には、自動で少し間が空きます。

ブラウザの中では、次のような「木」になります。

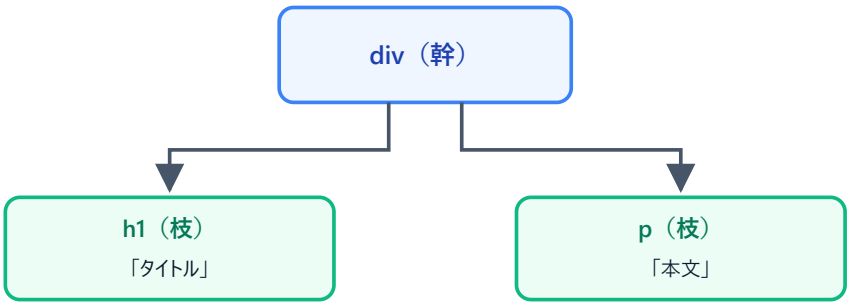


図3-1: HTML のタグの入れ子は、木のような形（ツリー）になる。div が幹、h1 と p が枝

家系図や組織図と同じで、**親（parent）・子（child）・兄弟（sibling）** の関係があります。上の例では `div` が親、`h1` と `p` が子、`h1` と `p` どうしは兄弟です。

なぜ「木」にするの？ 木の形にしておくと、プログラム（JavaScript）が「この `h1` の文字を変えて」「この `div` の中に新しい `p` を足して」と、ピンポイントで操作できるからです。Reactは、まさにこのDOMを **JavaScriptで効率よく書き換える** ことで、ページ全体を読み込み直さずに画面の一部だけを更新します（第5章で詳述）。

0.3 3つの言語の役割（再確認）

種類	役割	家でたとえると	本アプリの該当ファイル
HTML	構造・内容（何があるか）	骨組み（柱・壁・部屋割り）	`.tsx` の中のJSX
CSS	見た目（どう見えるか）	内装・塗装・家具の配置	`styles/` の中の `.css`
JavaScript / TypeScript	動き（何が起きるか）	電気・スイッチ・設備	`.tsx` の処理部分

この章はHTMLとCSSに集中します。JavaScript / TypeScript は第4章・第5章で扱います。

> **用語: TypeScript（タイプスクリプト）とは？** JavaScript（ジャバスクリプト）に「型（かた）」というルールを足した言語。本アプリのフロントは `.tsx` という拡張子のファイルで、これは「TypeScript + JSX」という意味です。詳しくは第4章。

---

1. HTMLの基本

> **HTML（エイチティーエムエル：HyperText Markup Language）とは？**「**タグ**」という印で文書に意味のラベルを貼り、構造を表す言語です。「Markup（マークアップ）」は「印を付ける」という意味。文章のどこが見出しで、どこが段落で、どこがボタンなのかを、タグで示します。

1.1 タグの形 — 開始・中身・終了

コピー

```
<p>これは段落です。</p>
```

分解すると：

- `<p>` … **開始タグ**（「ここから段落（paragraph）」の合図）
- `これは段落です。` … **中身**（**コンテンツ**）
- `</p>` … **終了タグ**（スラッシュ `/` が付く。「段落ここまで」の合図）

> **たとえ:** タグは「付箋」のようなものです。`<p>` という付箋を貼り始め、`</p>` で貼り終わる。挟まれた範囲が「段落」という意味になります。

タグは **入れ子（ネスト）** にできます。外側のタグの中に内側のタグを入れます。

コピー

```
<div>                  <!-- 外側の箱 -->
  <h1>見出し</h1>       <!-- 内側：見出し -->
  <p>本文</p>           <!-- 内側：段落 -->
</div>                 <!-- 外側の箱おわり -->
```

> **▶ ここで初めて出てくるコード**
>
> - `<!-- -->`（HTML）: HTML では、`<!--` から `-->` までがコメントで、画面には出ません。

用語: コメント `<!-- ... -->` HTMLの中に書いても画面に出ない「メモ書き」。人間向けの注釈です。上の `<!-- 外側の箱 -->` がそれです。

⚠️ **入れ子のルール:** 開始と終了は **必ず対応** させ、交差させてはいけません。`<a><b></a></b>` は誤り（aとbが交差している）、`<a><b></b></a>` が正しい（bがaの中に完全に収まっている）。交差させると、ブラウザが意図しない形に直してしまい、表示が崩れます。

1.2 空要素（終了タグがないタグ）

中身を持たないタグは、終了タグがありません。これを **空要素（くうようそ）** と呼びます。

コピー

```
<img src="logo.png" alt="ロゴ">   <!-- 画像。中身なし -->
<input type="text">               <!-- 入力欄。中身なし -->
<br>                              <!-- 改行 -->
<hr>                              <!-- 水平線 -->
```

▶ ここで初めて出てくるコード

- **`<img>`**（HTML）: 画像を表示するタグ。`<img src="logo.png" alt="ロゴ">` のように、`src` に画像の場所、`alt` に画像の説明を書きます。中身を持たないので、閉じタグ（`</img>`）は書きません。
- **`<input>`**（HTML）: 1 行の入力欄のタグ。`type` で種類（`text`・`email`・`password`・`hidden` など）を、`name` で送る時の名前を決めます。閉じタグはありません。
- **`<br>`**（HTML）: 改行のタグ。閉じタグはありません。段落を分けたい時は、`<br>` を並べるより `<p>` を分けます。
- **`<hr>`**（HTML）: 横線（水平線）を引くタグ。話の区切りに使います。中身を持たないので、閉じタグは書きません。
- **`src=`**（HTML）: `src` は、読み込むファイル（画像・スクリプト）の場所です。
- **`alt=`**（HTML）: `<img alt="ロゴ">` の `alt` は、画像の代わりの説明の文。画像が表示できない時や、読み上げソフトで読む時に使われます。
- **`type=`**（HTML）: `<input type="…">` の `type` は、入力欄の種類です。`text`（文字）・`email`・`password`（伏せ字）・`number`・`date`・`checkbox`（チェック）・`file`（ファイル）・`search` などがあります。`<button type="submit">` は、押すとフォームを送るボタンです。

なぜ終了タグがないの？ これらは「中に文章を挟む」必要がないからです。画像や入力欄は、それ自体が1つの部品で、間に何かを入れる余地がありません。

⚠️ **JSX（React）では空要素も必ず閉じる:** Reactの世界では、空要素も末尾に `/` を付けて `<img />` `<input />` のように「自己完結」させる必要があります。これを忘れるとエラーになります（§3で実例を見ます）。

1.3 属性（attribute）

タグに追加情報を与えるのが **属性（attribute：アトリビュート）**。`属性名="値"` の形で **開始タグの中** に書きます。

コピー

```
<a href="/help" target="_blank" class="link">ヘルプ</a>
<!--  href=リンク先   target=開き方   class=スタイル用の名札   -->
```

▶ ここで初めて出てくるコード

- **`<a>`**（HTML）：リンクのタグ。`<a href="URL">文字</a>` で、押すとその URL へ移ります。
- **`href=`**（HTML）：`href` は、リンク先や、読み込むファイルの場所（URL）です。`<a href="/help">ヘルプ</a>` は、押すと `/help` へ移るリンクです。
- **`target=`**（HTML）：`<a href="/help" target="_blank">` の `target="_blank"` は「リンク先を新しいタブで開く」という指定です。
- **`class=`**（HTML）：`class` は、タグに付ける「グループの名前（名札）」です。CSS で `.blue_button { … }` と書くと、`class="blue_button"` の物全部に、同じ見た目が付きます。空白で区切って、いくつも付けられます。

代表的な属性：

属性	意味	例
`id`	要素に1つだけ付ける固有の名札	`id="numberInput"`
`class`	スタイル用の名札（複数要素で共有可）	`class="blue_button"`
`href`	リンク先URL（`<a>` 用）	`href="/login"`
`src`	読み込むファイルの場所（画像など）	`src="logo.png"`
`alt`	画像が出ないときの代替文字	`alt="ロゴ"`
`type`	入力欄やボタンの種類	`type="number"`
`value`	入力欄の値	`value="123"`
`required`	入力必須（値なし属性）	`required`
`disabled`	操作不可（値なし属性）	`disabled`
`role`	要素の「役割」を支援技術に伝える	`role="alert"`

用語: id と class の違い `id`（アイディー）は「1ページに1つだけ」の固有名（マイナンバーのようなもの）。`class`（クラス）は「同じ見た目を複数の要素に付ける」ための共有名（制服のようなもの）。CSSでは `id` は先頭に `#`、`class` は先頭に `.` を付けて指定します（§4）。

用語: 値なし属性（boolean属性） `required` や `disabled` は値を書かず、書くだけで「オン」になります。`required` と書けば「必須」、書かなければ「任意」。スイッチのようなものです。

1.4 よく使うタグ早見表

コピー

```
<h1>〜<h6>     <!-- 見出し（h1が最大、h6が最小） -->
<p>            <!-- 段落（paragraph） -->
<div>          <!-- 意味を持たない箱（レイアウト用）。最頻出 -->
<span>         <!-- 行内の小さな箱（文章の一部だけを囲む） -->
<a href="">    <!-- リンク（anchor） -->
<img src="">   <!-- 画像（image） -->
<ul><li>       <!-- 順序なしリスト（箇条書き） -->
<ol><li>       <!-- 順序ありリスト（番号付き） -->
<table>        <!-- 表 -->
  <thead><tbody>  <!-- 表のヘッダ部・本体部 -->
  <tr>            <!-- 行（table row） -->
  <th>            <!-- 見出しセル（table header） -->
  <td>            <!-- データセル（table data） -->
<nav>          <!-- ナビゲーション（メニュー）の領域 -->
<form>         <!-- 入力フォームの枠 -->
<label>        <!-- 入力欄の見出しラベル -->
<input>        <!-- 入力欄 -->
<button>       <!-- ボタン -->
<select><option>  <!-- ドロップダウン選択 -->
<textarea>     <!-- 複数行の入力欄 -->
```

▶ ここで初めて出てくるコード

- `<h6>`（HTML）：いちばん小さい見出しのタグ。見出しは `<h1>`（いちばん大きい）から `<h6>`（いちばん小さい）まで 6 段階あります。
- `<span>`（HTML）：文の一部分を囲む、特別な意味の無いタグ。CSS で、その部分だけ色を変える時などに使います（ `<div>` と違い、改行しません）。
- `<ul>`（HTML）：順番の無い並び（点の付いた並び）のタグ。中に `<li>` を並べます。
- `<li>`（HTML）：並び（ `<ul>`・`<ol>` ）の 1 つのタグ。
- `<ol>`（HTML）：番号の付いた並びのタグ。中に `<li>` を並べると、1. 2. 3. と番号が付きます（手順など）。
- `<table>`（HTML）：表のタグ。中に `<thead>`（見出しの行）と `<tbody>`（中身の行）を入れ、行は `<tr>` 、ますは `<th>`（見出し）か `<td>`（中身）で作ります。
- `<thead>`（HTML）：表の見出しの行をまとめるタグ。
- `<tbody>`（HTML）：表の中身の行をまとめるタグ。
- `<tr>`（HTML）：表の 1 行（table row）のタグ。中に `<th>` や `<td>` を並べます。
- `<th>`（HTML）：表の見出しのます（table header）のタグ。ふつうは太字で表示されます。
- `<td>`（HTML）：表の中身のます（table data）のタグ。
- `<nav>`（HTML）：メニュー（ほかのページへのリンクのまとまり）を表すタグ。
- `<form>`（HTML）：入力欄とボタンをまとめて、送る（サーバーへ届ける）ためのタグ。`action` に送り先、`method="post"` に送り方を書きます。
- `<label>`（HTML）：入力欄の名前を表すタグ。`<label for="numberInput">従業員番号</label>` の `for` と、入力欄の `id` を同じにすると、名前を押しても入力欄が選ばれ、読み上げソフトにも伝わります（React では `for` の代わりに `htmlFor` と書きます）。
- `<button>`（HTML）：ボタンのタグ。フォームの中の `<button type="submit">送る</button>` は、押すとフォームを送ります。`type="button"` にすると、送らない、ふつうのボタンになります。
- `<select>`（HTML）：選ぶ欄（プルダウン）のタグ。中に `<option value="値">表示する文字</option>` を並べます。`selected` を付けた物が、最初に選ばれます。
- `<option>`（HTML）：`<select>`（選ぶ欄）の中の、選べる 1 つ。`<option value="new">新しく作る</option>` の `value` が送られる値、間の文字が画面に出る文字です。`selected` で最初に選ばれます。
- `<textarea>`（HTML）：何行も書ける入力欄のタグ。`<textarea name="detail" rows="4"></textarea>` の `rows` は、見えている行の数です。最初から入れておく文字は、タグの間に書きます。

用語: セマンティックHTMLとは？「セマンティック（semantic）」は「意味のある」という意味。`<div>` ばかり使うのではなく、`<nav>`（メニュー）・`<table>`（表）・`<button>`（ボタン）のように **役割の分かるタグ** を使うこと。検索エンジンや、目の不自由な方が使う **読み上げソフト** が、内容を正しく理解できるようになります。本アプリも `<nav>` `<table>` `<form>` `<button>` を適切に使っています（§11・§13で確認）。

div と span の違い: どちらも「意味を持たない箱」ですが、`<div>` は **ブロック要素**（縦に積み上がり、横幅いっぱいに広がる）、`<span>` は **インライン要素**（文章の途中に置け、必要な分だけ幅をとる）。たとえ話: `div` は「段落まるごとを囲む大きな袋」、`span` は「文章中の一単語だけにマーカーを引く」イメージ。

---

2. HTMLフォーム（本アプリの心臓）

本アプリは「工数入力」「人員登録」「ログイン」「問い合わせ」など **入力フォームの塊** です。ここを制すれば本アプリのHTMLの大半が読めます。

用語: フォーム（form：フォーム）とは？ ユーザーが文字や数値を入力し、サーバーへ送るための「申込用紙」のようなまとまり。入力欄・ラベル・送信ボタンをひとまとめにします。

2.1 フォームの基本構造

▼ このコードがやること（先に日本語で）:「従業員番号」というラベルの付いた数値入力欄と、「ログイン」という送信ボタンを、1つのフォーム（申込用紙）にまとめています。

コピー

```
<form>                                          <!-- 入力をまとめる枠。送信の単位 -->
  <label for="numberInput">従業員番号</label>    <!-- 見出しラベル -->
  <input type="number" id="numberInput">        <!-- 入力欄 -->
  <button type="submit">ログイン</button>        <!-- 送信ボタン -->
</form>
```

▶ ここで初めて出てくるコード

- **`for=`**（HTML）: `<label for="q">` の `for` は、この説明の文字が、どの入力欄（`id="q"`）のための物かを結び付けます。文字を押しても、その欄に入力できるようになり、読み上げソフトも欄の名前を読めます。
- **`id=`**（HTML）: `id` は、ページの中で 1 つだけの名前です。`<label for="title">` で欄と結び付けたり、`#title` のリンクでその場所へ飛んだりできます。同じ `id` を 2 つ付けてはいけません。

- `<form>` … 入力欄をまとめる枠。中の送信ボタンを押すと「送信（submit）」というイベント（出来事）が起きます。
- `<label for="ID">` … 入力欄の見出し。`for` を `<input>` の `id` と **一致** させると、ラベルの文字をクリックしただけで、その入力欄にカーソルが入ります（クリックしやすくなる）。
- `<button type="submit">` … フォームを送信するボタン。

2.2 input の type（種類）一覧

`type` を変えると、入力欄の見た目と挙動がガラッと変わります。本アプリで使うものを中心に：

type	用途	見た目・挙動	本アプリでの例
`text`	一般の文字	ふつうの一行入力	氏名など
`number`	数値	右端に上下ボタンが付く	従業員番号
`date`	日付	カレンダーが開く	就業日・検索日
`checkbox`	オン/オフ	四角いチェックボックス	権限・各種フラグ
`password`	伏せ字	入力が ●●● になる	（必要に応じて）
`radio`	複数から1つ選ぶ	丸いラジオボタン	選択肢

> ⚠️ **`<input type="number">` の注意:** カレンダー型や数値型でも、入力された値はJavaScript側では **文字列** として届きます。だから `Login.tsx` では `Number(employee_no)` と数値に変換しています（§3⑩、詳しくは第4章）。「画面の数字」と「プログラムが受け取る数字」は別物だと意識してください。

2.3 select（ドロップダウン）と textarea（複数行入力）

> ▼ **このコードがやること:** 上は「PかRを選ぶドロップダウン」、下は「複数行の文章を入れる欄」です。

コピー

```
<select>                          <!-- ドロップダウン選択 -->
  <option value="P">P</option>    <!-- 選択肢1 -->
  <option value="R">R</option>    <!-- 選択肢2 -->
</select>

<textarea rows="4"></textarea>    <!-- 複数行入力（問い合わせ本文など）。rows=表示する行数 -->
```

▶ ここで初めて出てくるコード

- `value=`（HTML）：`value` は、入力欄の最初の値や、選び肢（`<option>`）を選んだ時に送られる値です。
- `rows=`（HTML）：`<textarea rows="6">` の `rows` は、見える行の数です。

本アプリのショップ選択（`ShopSelect`）や問い合わせ入力（`InquirNew`）で使われています。`<option value="...">` の `value` が「実際に送られる値」、タグの中身が「画面に見える文字」です。

---

3. 実例：Login.tsx のJSXを1行ずつ完全解説

ここで、本アプリのログイン画面 `frontend/src/MainPage/Login.tsx` を、HTMLの知識で読みます。このファイルは「画面を作る部分（JSX）」と「動きを作る部分（JavaScript）」が混ざっています。本章では **画面部分** を中心に、1行ずつ追います（動きの部分は第4章・第5章で深掘り）。

用語: JSX（ジェイエスエックス）とは？ JavaScript（TS）の中に、HTMLのような記述を混ぜて書く書き方。Reactで画面を書く記法です。HTMLとの主な違いは次の3点： - `class` → `className`（「class」はJavaScriptで別の意味の予約語なので避ける） - `for` → `htmlFor`（同様に「for」も予約語） - `{ }` の中に **JavaScriptの値や処理** を埋め込める

3.1 ファイル冒頭の import（読み込み）部分

▼ **このコードがやること:** この画面で使う部品（React本体、ページ移動の道具、通信の道具、ロゴ画像、専用CSS）を外部から読み込んでいます。「材料を仕入れる」段階です。

コピー

```
// ① React本体と、状態を持つ道具 useState
import React, { useState } from "react";
// ② 画面間リンクと、画面移動の道具
import { Link, useNavigate } from "react-router-dom";
// ③ サーバーと通信する道具（第4章）
import api from "../api/axios";
// ④ 通信ライブラリ本体
import axios from "axios";
// ⑤ ロゴ画像を読み込んで logo という名前に
import logo from "../img/TitleRogo.png";
// ⑥ この画面専用のCSS（§12で全解説）
import styles from "../styles/MainPage/Login.module.css";
```

- ⑤ … 画像も `import` で読み込みます。後で `<img src={logo} />` のように使います。
- ⑥ … `Login.module.css` を `styles` という名前で読み込み。`styles["login-wrapper"]` のように、CSSのクラスを取り出して使います（**CSS Modules**。§14で詳述）。

3.2 画面を作って返す部分（JSX本体）

> **▼ このコードがやること（先に日本語で）**：画面全体を `login-wrapper` という箱で囲み、その中に「ロゴ画像」「ログインフォーム（見出し・従業員番号の入力欄・エラー表示・ログインボタン）」「ヘルプボタン」を縦に並べています。これがログイン画面の見た目の正体です。

コピー

```
return (
  {/* ① 画面全体を囲む箱。Modulesのクラスを適用 */}
  <div className={styles["login-wrapper"]}>
    {/* ② ロゴ画像。src=画像、alt=代替文字 */}
    <img src={logo} alt="業務工数システムロゴ"
         className={styles["login-logo"]} />
    {/* ③ フォーム枠（申込用紙） */}
    <form
      {/*    送信時に handleSubmit を呼ぶ（第4章） */}
      onSubmit={handleSubmit}
      {/*    キー入力時の処理（Enterの暴発防止） */}
      onKeyDown={(e) => {
        if (e.key === "Enter" && e.target instanceof HTMLInputElement
            && e.target.type !== "textarea") {
          {/*    Enterでの誤送信を止める */}
          e.preventDefault();
          {/*    入力欄からフォーカス（カーソル）を外す */}
          (e.target as HTMLInputElement).blur();
        }
      }}
    >
      {/* ④ 見出し（global.cssで中央寄せ） */}
      <h2>ログイン</h2>
      {/* ⑤ 入力欄をまとめる箱（縦並び） */}
      <div className={styles["search-bar"]}>
        {/* ⑥ ラベル。htmlForを下のidと一致 */}
        <label htmlFor="numberInput">従業員番号</label>
        <input
          {/* ⑦ 数値入力欄 */}
          type="number"
          {/*    ラベルと結ぶ id */}
          id="numberInput"
          {/*    表示する値（stateと連動。第5章） */}
          value={employee_no}
          {/*    入力が変わるたびに */}
          onChange={(e) => {
            const value = e.target.value;
            {/*    stateを更新（空なら空、なければ値） */}
            setNumber(value === "" ? "" : value);
          }}
          {/* ⑧ 入力必須 */}
          required
          className={styles["input-focus"]}
        />
        {/* ⑨ エラーがあるときだけ表示（&&） */}
        {errorMessage && (
          {/*    role="alert"は支援技術への通知（§16） */}
          <div role="alert">{errorMessage}</div>
```

```
        )}

        {/* ⑩ 送信ボタン。global.cssの青ボタン */}
        <button type="submit" className="blue_button">ログイン</button>
      </div>
    </form>
    {/* ⑪ ヘルプへのリンク（菱形ボタン。§12） */}
    <Link to="/help" className={styles["help-button"]}></Link>
  </div>
);
```

各番号の解説：

- ①⑤ … `styles["..."]` は **CSS Modules**（§14）。`Login.module.css` のクラスを当てています。`styles["login-wrapper"]` は「Login.module.css の `.login-wrapper` ルールを適用せよ」という意味です。
- ② … `<img />` は空要素なので末尾を `/>` で閉じます。`src={logo}` の `{ }` は「JavaScriptの値（先ほど import した logo 画像）を入れる」記法。`alt="業務工数システムロゴ"` は、画像が表示できないときや読み上げ時の代替テキスト（§16のアクセシビリティ）。
- ③ … `<form>` に `onSubmit`（送信時の処理）と `onKeyDown`（キー入力時の処理）を結びつけています。`on○○` で始まる属性は「○○が起きたら、この処理を実行せよ」という **イベントハンドラ**（第5章で詳述）。
- ④ … `<h2>` は見出し。`global.css` の `h2` ルールで中央寄せ＆サイズ指定されます（§11.6）。
- ⑥ … `htmlFor="numberInput"` を、下の `<input>` の `id="numberInput"` と一致させています。これで「従業員番号」の文字をクリックすると入力欄にカーソルが入ります。HTMLの `for` がJSXでは `htmlFor` になる実例です。
- ⑦⑧ … `type="number"` で数値入力欄、`required` で入力必須に。
- ⑨ … `{errorMessage && (<div role="alert">...)}` は「`errorMessage` に中身があるときだけ、後ろの要素を表示」という条件表示（第5章§2）。エラー時だけ赤いアラート（§11.11で全解説）が出ます。`role="alert"` は読み上げソフトへの「これは警告です」という合図。
- ⑩ … `className="blue_button"` （**文字列で直接**）は **global.css**（§11）の共通青ボタン。同じ画面で「Modules（`styles[...]`）」と「global（文字列）」が混在しているのがポイントです。
- ⑪ … `<Link to="/help">` はReact Router（第5章）の画面内リンク。見た目は `help-button` クラス（45度傾いた菱形。§12で全解説）。中身が空なのは、文字ではなくCSSの `::after` で「?」を描いているからです。

> **このように、1つのログイン画面でも「JSX（HTML構造） ＋ CSS Modules ＋ global.css ＋ Reactの処理」が組み合わさっています。**次は、この見た目を作っているCSSを徹底的に読みます。

---

4. CSSの基本文法

> **用語: CSS（シーエスエス：Cascading Style Sheets）とは？** HTMLの **見た目** を指定する言語。「Cascading（カスケーディング）」は「滝のように上から流れて重なる」という意味で、複数の指示が重なったときの優先順位のしくみを表します（後述）。

4.1 ルールの形

コピー

```
セレクタ {
  プロパティ: 値;
  プロパティ: 値;
}
```

具体例（※説明用の簡易例）：

コピー

```
/* セレクタ：対象（すべての p） */
p {
  /* プロパティ:値 → 文字色を赤に。最後は必ず ; */
  color: red;
  /* 文字サイズ16ピクセル */
  font-size: 16px;
}
```

> **▶ ここで初めて出てくるコード**
>
> - **`color:`**（CSS）: 文字の色を決める項目。`color: red;` や `color: #333;`、`color: var(--main-color);`（変数の色）のように書きます。
> - **`font-size:`**（CSS）: CSS で、文字の大きさを決める項目。
> - **`/* */`**（CSS）: CSS では、`/*` から `*/` までがコメントです。
> - **`数px`**（CSS）: CSS の `16px` の `px` は、画面の点（ピクセル）の数で表す大きさの単位です。

- **セレクタ**（selector） … 「誰に」スタイルを当てるかの指定（上の例では「すべての `<p>`」）。
- **プロパティ**（property） … 「何を」変えるか（色、サイズ、余白…）。
- **値**（value） … 「どう」するか（赤、16px…）。
- 各行の末尾には **必ずセミコロン** `;` を付けます。これを忘れると、その行から次の行までが壊れます（よくあるミス）。

4.2 セレクタ（対象の指定）一覧

書き方	意味	例
`p`	タグ名で指定	すべての `<p>`
`.box`	クラスで指定（先頭ドット）	`class="box"` の要素
`#header`	IDで指定（先頭シャープ）	`id="header"` の要素
`.box p`	子孫（box の中のすべての p）	
`.box > p`	直接の子のみ	
`a:hover`	擬似クラス（マウスを乗せた時）	§8
`input[type="date"]`	属性で指定（属性セレクタ）	§11で実例
`div[role="alert"]`	属性の値まで指定	§11.11で実例
`*`	すべての要素（全称セレクタ）	§11.1で実例
`a, p`	複数まとめて（カンマ区切り）	
`th, td`	表のヘッダとデータ両方	§11.9で実例

4.3 カスケード（優先順位）— 同じ要素に複数のルールが当たったら

> **用語: カスケード（cascade：カスケード）とは？** CSSの「C」。複数のルールが同じ要素に同時に当たったとき、**優先順位** に従って勝者が決まるしくみ。

優先順位はおおまかに次の順です（強い順）：

1. `!important` を付けたもの（最優先・乱用禁物）
2. インラインスタイル（`style="..."` で直接書いたもの）
3. `#id` で指定
4. `.class` や `[属性]` で指定
5. タグ名で指定

そして **同じ強さなら「後に書いた方」が勝ちます。**

> ⚠️ **`!important` の乱用に注意:** `!important` を付けると最優先になりますが、これを多用すると「なぜか上書きできないルール」だらけになり、保守が地獄になります。本アプリでも `MainMenu.module.css` の `display: none !important;`（§13）など、ごく限られた場所でのみ使われています。

4.4 CSSの書き場所（3通り）

1. **外部ファイル**（推奨・本アプリの方式）：`.css` ファイルに書き、`import` で読み込む。
2. **インラインスタイル**：`<div style="color:red">`（その要素だけ。Reactでは `style={{color:'red'}}`）。
3. `<style>` **タグ**：HTML内に直接。本アプリでは基本使いません。

本アプリは **外部ファイル**（`global.css` と各 `.module.css`）です。理由は「見た目をまとめて管理でき、複数画面で使い回せる」から。

---

5. ボックスモデル（余白の正体）

すべてのHTML要素は、**四角い箱** として扱われます。この箱の構造を「ボックスモデル」と呼び、CSSで最も重要な概念の1つです。「なぜか余白が消えない」「なぜか幅がはみ出す」の9割は、ここの理解不足が原因です。



図3-2: ボックスモデル。中身（content）の外に、内側の余白（padding）・枠線（border）・外側の余白（margin）が重なる

- **content（コンテンツ）** … 中身（文字や画像）。`width`・`height` で大きさを指定。
- **padding（パディング）** … 枠の内側の余白。中身と枠の間。「箱の内張り」。
- **border（ボーダー）** … 枠線。
- **margin（マージン）** … 枠の外側の余白。他の箱との距離。「箱と箱のソーシャルディスタンス」。

> **覚え方:** 内側から「中身→内張り(padding)→壁(border)→外の距離(margin)」。`padding` は「中身を枠から離す」、`margin` は「箱を他の箱から離す」。

5.1 box-sizing: border-box が超重要

> ▼ **なぜ重要か（先に日本語で）**：既定では `width`（幅）は「中身だけの幅」を指し、padding と border は **その外側に追加** されます。つまり `width:200px; padding:10px` だと、見た目の実寸は 200 + 10 + 10 = 220px になり、計算が狂います。

これを防ぐのが `box-sizing: border-box`。これを指定すると `width` が「padding と border を **含めた** 全体幅」になり、`width:200px` と書けば見た目もちゃんと200pxになります。直感どおりになるわけです。

コピー

```
/* 全要素を「幅にpadding/borderを含める」方式に（※本アプリの実装。§11.1で再掲） */
* { box-sizing: border-box; }
```

> ▶ **ここで初めて出てくるコード**
>
> - **`box-sizing:`**（CSS）：CSS で、箱の幅に枠線と内側の余白を含めるかを決める項目。`box-sizing: border-box;` にすると、`width` の中に `padding` と `border` が含まれ、大きさの計算が分かりやすくなります。
> - **`*`**（CSS）：CSS の `*` は、「全部のタグ」を選ぶ書き方です。`* { box-sizing: border-box; }` は、全部のタグの幅の数え方をそろえます。

本アプリは `global.css` の冒頭で **全要素に** これを適用しています（§11.1で実コード確認）。だから本アプリでは「幅の計算が狂う」事故が起きにくくなっています。

5.2 margin / padding のショートハンド（まとめ書き）

コピー

```
/* 上下左右すべて10px */
margin: 10px;
/* 上下10px / 左右20px */
margin: 10px 20px;
/* 上→右→下→左（時計回り） */
margin: 10px 20px 30px 40px;
/* 上下0、左右auto＝左右中央寄せ（頻出！） */
margin: 0 auto;
/* 上だけ個別指定 */
margin-top: 8px;
```

▶ ここで初めて出てくるコード

- **`margin:`**（CSS）: 箱の外側の余白を決める項目。`margin: 12px;` は上下左右に 12px。2 つ書くと「上下 左右」、4 つ書くと「上 右 下 左」の順です。
- **`margin-top:`**（CSS）: CSS で、箱の上の外側の余白を決める項目。

`margin: 0 auto` は「横方向の中央寄せ」の定番。左右を `auto`（自動）にすると、ブラウザが左右の余白を等しく取り、結果として要素が水平中央に来ます。本アプリの `login-logo` や `form`、各種 `wrapper` で多用されています（§11・§12）。

---

6. 単位と色

6.1 大きさの単位

単位	意味	使いどころ
`px`	ピクセル（絶対的な点）	枠線・細かい余白
`%`	親要素に対する割合	幅を親の何%にするか
`rem`	ルート（html）の文字サイズ基準	文字サイズ（拡大縮小に強い）
`em`	その要素の文字サイズ基準	アイコンを文字に合わせる等
`vw` / `vh`	画面幅 / 高さの1%（viewport）	画面サイズに応じた大きさ

用語: viewport（ビューポート）とは？「今ブラウザで見えている表示領域」のこと。`vw` は viewport width（表示幅）の1%、`vh` は viewport height（表示高さ）の1%。`100vh` なら「画面の高さいっぱい」。本アプリは `min-height: 100vh`（最低でも画面の高さ分は確保）で使っています（§11.6）。

6.2 便利な関数 — clamp / calc / min

本アプリは画面サイズへの対応のため、CSSの関数を多用します。

clamp(最小, 推奨, 最大) …「推奨値を使うが、最小〜最大の範囲に収める」関数。画面サイズに応じて文字を伸縮させつつ、小さすぎ・大きすぎを防ぎます。本アプリは見出しサイズに使っています（§11.5）：

コピー

```
--heading1: clamp(1.5rem, 1.5rem + 3vw, 4rem);
/* 最小1.5rem、画面幅に応じて1.5rem+3vw、最大4rem */
```

calc(式) … CSSの中で計算ができる関数。`calc(100% - 60px)` なら「親の幅いっぱいから60px引いた値」。本アプリのログイン入力欄の幅に使っています（§12.4）。

min(値A, 値B) …「2つのうち小さい方」を採る関数。`min(400px, 90%)` なら「大画面では400px、小画面では画面の90%」。ロゴのはみ出し防止に使っています（§12.3）。

6.3 色の指定

コピー

```
/* 色名（限られた数） */
color: red;
/* 16進数 #RRGGBB（R赤 G緑 B青 を00〜ffで） */
color: #007bff;
/* 3桁省略形（#00ff00 と同じ＝緑） */
color: #0f0;
/* 赤緑青を0〜255で */
color: rgb(0, 123, 255);
/* 最後は透明度(0=透明〜1=不透明) */
color: rgba(0, 0, 0, 0.1);
```

▶ ここで初めて出てくるコード

- **`rgb()`**（CSS）：色を「赤・緑・青」の強さ（0〜255）で書く書き方。`rgb(0, 123, 255)` は青っぽい色です。
- **`rgba()`**（CSS）：`rgb()` に、透けぐあい（0＝透明〜1＝不透明）を足した書き方。`rgba(0, 0, 0, 0.1)` は、薄く透けた黒（影などに使う）です。
- **`#rrggbb`**（CSS）：CSS の `#0f172a` のような `#` と 6 けた（または 3 けた）の 16 進数は、色の書き方です。2 けたずつ、赤・緑・青の強さ（00〜ff）を表します。`#ffffff` は白、`#000000` は黒です。

16進数カラーの読み方: `#007bff` は「赤=00、緑=7b、青=ff」。`#0f0` は短縮形で `#00ff00`（赤0・緑最大・青0＝純緑）と同じ。本アプリは主に **16進数**（`#007bff` 青、`#0f0` 緑、`#0ff` 水色 など）を使い、影の透明度には **rgba** を使っています（§11）。

7. Flexbox（配置の主役）

要素を **横並び・縦並び・中央寄せ** するための、現代CSSの主役です。本アプリのレイアウトはほぼすべてFlexboxで作られています。

> **用語: Flexbox（フレックスボックス）とは？**「Flexible Box Layout（柔軟な箱配置）」の略。親要素に `display: flex` を付けると、その中の子要素を自由に並べられるしくみ。「子要素を整列させる魔法のトレイ」とイメージすると分かりやすいです。

7.1 基本

親要素に `display: flex` を付けると、その **直接の子要素** が「フレックスアイテム」として並びます。

コピー

```
.container {
  /* フレックス開始（この箱の子を整列させる） */
  display: flex;
  /* 並ぶ向き：row=横（既定）, column=縦 */
  flex-direction: row;
  /* 主軸（並ぶ向き）の揃え */
  justify-content: center;
  /* 交差軸（直角方向）の揃え */
  align-items: center;
  /* アイテム間の隙間 */
  gap: 15px;
  /* 入りきらない時に折り返す */
  flex-wrap: wrap;
}
```

▶ ここで初めて出てくるコード

- **`display:`**（CSS）：箱の並べ方を決める項目。`display: flex;` は「中身を横一列に並べる」、`display: grid;` は「ます目に並べる」、`display: none;` は「表示しない」です。
- **`flex-direction:`**（CSS）：`display: flex` の中身を並べる向きを決める項目。`row` は横（最初の値）、`column` は縦に並べます。
- **`justify-content:`**（CSS）：`display: flex` の中身を、並ぶ向きにどう配るかを決める項目。`space-between` は「両端に寄せて、間を均等に空ける」。
- **`align-items:`**（CSS）：`display: flex` の中身を、上下（横に並べた時の縦）のどこにそろえるかを決める項目。`center` で真ん中にそろえます。
- **`gap:`**（CSS）：`flex` や `grid` の中身どうしの間の幅を決める項目。`gap: 12px 16px;` は「行の間 12px、列の間 16px」。
- **`flex-wrap:`**（CSS）：`display: flex` の中身が入りきらない時に、次の行へ折り返すかを決める項目。`wrap` で折り返します。
- **`.クラス名`**（CSS）：CSS の `.blue_button { … }` のように `.` で始まる名前は、「`class="blue_button"` の付いたタグ全部」を選ぶ書き方（クラスセレクター）です。`{ }` の中の見た目が、その全部に付きます。

7.2 justify-content と align-items の値

- **`justify-content`（主軸方向の揃え）**：`flex-start`（先頭寄せ）/ `center`（中央）/ `flex-end`（末尾寄せ）/ `space-between`（両端＋等間隔）/ `space-around`（各アイテムの周りに等間隔）。
- **`align-items`（交差軸方向の揃え）**：`stretch`（既定・引き伸ばし）/ `center`（中央）/ `flex-start` / `flex-end`。

覚え方（重要）：`flex-direction: row`（横並び）のとき、`justify-content` は **横方向** の揃え、`align-items` は **縦方向** の揃え。`column`（縦並び）にすると、この2つの担当方向が **入れ替わります**。「中央寄せできない！」の原因は、たいていこの軸の取り違えです。

本アプリの `search-bar`（ログインの入力エリア）は、`column`（縦並び）＋中央寄せ＋隙間15pxで作られています（§12.2で実コード確認）。一方、一覧画面の `KosuList` の `search-bar` は `row`（横並び）です（§13）。同じ名前でも画面ごとに方向が違うのが面白いところです。

8. 擬似クラスと擬似要素

8.1 擬似クラス（状態に応じたスタイル）

要素の「状態」に応じてスタイルを変えます。`:`（コロン）を **1つ** 付けます。

コピー

```
/* マウスを乗せている時 */
a:hover    { }
/* 入力欄が選択（カーソルが入っている）時 */
input:focus{ }
/* 偶数番目の行 */
tr:nth-child(even) { }
/* 操作不可の時 */
button:disabled { }
```

▶ ここで初めて出てくるコード

- **`button:`**（CSS）: `button:hover` は、「マウスを乗せている時のボタン」を指す CSS の書き方（`:hover` は乗せている時の状態）。
- **`:hover`**（CSS）: CSS の `button:hover` の `:hover` は、「マウスが乗っている間だけ」の見た目です。`:focus` は「キーボードで選ばれている間」、`:disabled` は「使えない時」です。
- **`:focus`**（CSS）: 入力欄などが「今、入力の場所になっている（カーソルがある）」時だけ当てるスタイル。`input:focus { … }`。
- **`:nth-child`**（CSS）: 「何番目の子か」で選ぶ書き方。`tr:nth-child(even)` は偶数番目の行、`odd` なら奇数番目の行です。表を 1 行おきに色分けする時に使います。
- **`:disabled`**（CSS）: 押せない（使えない）状態のボタンや入力欄にだけ当てるスタイル。`button:disabled { opacity: 0.5; }`。

用語: hover（ホバー） / focus（フォーカス） - `:hover` … マウスポインタを乗せている状態。ボタンの色変化などに使う。- `:focus` … 入力欄などが「選択されてキー入力を受け付ける」状態。枠を光らせる演出に使う。

本アプリは `:hover`（ボタンの色変化）と `:focus`（入力欄の枠を青く光らせる）を多用します（§11・§12・§13）。`:disabled` はページ送りの「これ以上戻れない」ボタンに使われています（§13のPagination）。

8.2 擬似要素（要素の一部を作る）

要素の前後に「飾りの中身」を追加します。`::`（コロン）を **2つ** 付けます。

なぞってコピーすると絵文字が別の文字になります。正しくは → コピー

```
/* 要素の中身の「前」に文字を足す */
.icon::before { content: "⚠️ "; }
/* 要素の中身の「後」に文字を足す */
.icon::after  { content: "?"; }
```

▶ ここで初めて出てくるコード

- `content:`（CSS）：`::before` や `::after` で足す中身。`content: "⚠️ ";` は文字を足し、`content: "";` は中身の無い飾り（形や色だけ）を作ります。
- `::before`（CSS）：部品の中身の前に、CSS で飾りの文字や形を足す書き方。`content: "⚠️ ";` で足す物を決めます。
- `::after`（CSS）：部品の中身の後ろに、CSS で飾りの文字や形を足す書き方。`content:` とセットで使います。

- `content` プロパティが **必須**（足す中身を指定。無いと表示されない）。
- 本アプリの **ヘルプボタンの「?」マーク**（§12.5）や、**アラートの「⚠️」**（§11.11）は、この `::after` / `::before` で作られています。HTMLには書かず、CSSだけで文字を「生やして」いるわけです。

---

9. トランジションとアニメーション

9.1 トランジション（なめらかな変化）

状態が変わるとき、見た目を **じわっと** 変化させます。

コピー

```
.blue_button {
  background-color: #007bff;
  /* 背景色を0.3秒かけて変化 */
  transition: background-color 0.3s ease;
}
.blue_button:hover {
  /* ホバーで濃い青へ（0.3秒かけてじわっと） */
  background-color: #0056b3;
}
```

▶ ここで初めて出てくるコード

- **`background-color:`**（CSS）：背景の色。`background-color: #007bff;` は青の背景です。
- **`transition:`**（CSS）：スタイルが変わる時に、少しずつ変える（なめらかに動かす）指定。`transition: background-color 0.3s ease;` は「背景の色を 0.3 秒かけて変える」。
- **`数s`**（CSS）：`0.3s` の `s` は秒（second）です。`transition` や `animation` の長さに使います。

- `transition: 対象プロパティ 時間 変化の緩急;` の形。
- `ease`（イーズ）は「最初ゆっくり→速く→最後ゆっくり」という自然な緩急。他に `linear`（一定速度）などがあります。
- これが無いと、ホバーの色変化が「パッ」と一瞬で切り替わります。`transition` があると「ふわっ」と気持ちよく変わります。本アプリの全ボタンに付いています（§11.10）。

9.2 キーフレームアニメーション（自動で動く）

`@keyframes`（キーフレーム）で「開始→終了」の変化を定義し、`animation` で再生します。

コピー

```
/* fadeInUp という名前のアニメを定義 */
@keyframes fadeInUp {
  /* 開始：透明＆50px下 */
  from { opacity: 0; transform: translateY(50px); }
  /* 終了：不透明＆定位置 */
  to   { opacity: 1; transform: translateY(0); }
}
.login-wrapper {
  /* 上のアニメを1秒で再生（下からふわっと出現） */
  animation: fadeInUp 1s ease-out;
}
```

▶ ここで初めて出てくるコード

- **`opacity:`**（CSS）：透明さを決める項目。`1` で不透明、`0` で見えない。`0.85` は少しだけ透ける（ボタンにマウスを乗せた時の合図など）。
- **`transform:`**（CSS）：部品を動かす・回す・大きくするなど、形を変える指定。`transform: translateY(50px);` は 50px 下へずらします。
- **`animation:`**（CSS）：`@keyframes` で決めた動きを部品に付けます。`animation: fadeInUp 1s ease-out;` は「`fadeInUp` の動きを 1 秒で、終わりをゆっくり」。
- **`@keyframes`**（CSS）：動き（アニメーション）の流れを名前を付けて決める書き方。`@keyframes fadeInUp { from { … } to { … } }` で、始め（from）と終わり（to）の様子を書きます。
- **`translateY()`**（CSS）：縦（Y の向き）にずらす書き方。`translateY(50px)` は下へ 50px、マイナスなら上へずらします。

- `opacity`（オパシティ） … 不透明度（0=透明〜1=不透明）。
- `transform: translateY(50px)` … 縦方向に50px移動（Y軸＝縦）。`translateX` なら横。
- `from` / `to` … アニメの「最初の状態」と「最後の状態」。
- 本アプリのログイン画面・一覧画面・メニュー画面は、これで **下からふわっと現れる** 演出をしています（§12・§13で実コード解説）。同じ `fadeInUp` が複数ファイルにコピーされているのも本アプリの特徴です。

---

10. メディアクエリ（レスポンシブ）

画面の幅に応じてスタイルを切り替え、PC・タブレット・スマホのいずれでも見やすくします。

コピー

```
/* 画面幅が768px以下のときだけ適用 */
@media (max-width: 768px) {
  form {
    width: auto;
    /* 最小幅の指定を解除（小さい画面で横はみ出し防止） */
    min-width: unset;
  }
}
```

▶ ここで初めて出てくるコード

- `max-width:`（CSS）：`@media (max-width: 640px) { … }` は、「画面の幅が 640px 以下の時だけ、中の決まりを使う」という書き方（スマホ向けの見た目）。箱に書く `max-width: 760px;` は、「幅の上限」です。
- `width:`（CSS）：CSS で、箱の幅を決める項目。
- `min-width:`（CSS）：いちばん小さい時の幅。これより細くはなりません。`unset` は「指定しない」に戻します。
- `@media`（CSS）：CSS の `@media (max-width: 640px) { … }` は、「画面の幅が 640px 以下の時だけ」の見た目を書く所です（メディアクエリ）。スマホでは 1 列にする、などに使います。
- `(max-width)`（CSS）：`@media` の `(max-width: 768px)` は、「画面の幅が 768px 以下」（スマートフォンや小さい画面）という条件です。`(min-width: …)` なら「以上」です。

- `@media (条件) { ... }` の中のルールは、条件に合うときだけ効きます。
- `max-width: 768px` は「画面幅が768px **以下**」。スマホやタブレット縦向きが該当します。
- `unset`（アンセット）は「その指定をなかったことにする」キーワード。
- 本アプリは `global.css`・`KosuList.module.css`・`MainMenu.module.css`・`Pagination.module.css` でこの768px分岐を使っています（各§で確認）。

用語: レスポンシブデザイン（responsive design）とは？ 1つのコードで、PCでもスマホでも見やすく表示を変える設計のこと。「responsive＝反応する」。画面サイズに「反応して」レイアウトが変わります。メディアクエリと、`clamp()` などの伸縮単位で実現します。

---

11. 実コード全解説：global.css 全342行を1行ずつ

ここからが本章の核心です。本アプリの **全画面共通のスタイル** `frontend/src/styles/global.css` を、**最初から最後まで** 読みます。これが読めれば、本アプリの見た目の「土台」を完全に理解できます。

11.1 全要素リセット（1〜5行目）

▼ **このブロックがやること:** すべての要素の初期の余白を消し、ボックスモデルを扱いやすい方式に統一します。「ブラウザごとの初期スタイルの違い」をならす、いわゆる **CSSリセット** です。

コピー

```
/* * は「すべての要素」を意味する全称セレクタ */
* {
  /* 全要素の外側余白を0に（ブラウザ既定の余白を消す） */
  margin: 0;
  /* 全要素の内側余白を0に */
  padding: 0;
  /* widthにpadding/borderを含める方式に統一（§5.1） */
  box-sizing: border-box;
}
```

▶ ここで初めて出てくるコード

- **`padding:`**（CSS）：箱の内側の余白（枠と中身の間）を決める項目。書き方は `margin` と同じです。

なぜリセットするの？ ブラウザは `<h1>` や `<p>` や `<body>` に、何も指定しなくても勝手に余白を付けます。しかもブラウザごとに微妙に違うため、まず全部0にしてから **自分で一から余白を設計** します。これがプロの定番手法です。`box-sizing: border-box` を全要素に効かせているので、本アプリは「padding を足したら幅がはみ出した」事故が起きにくくなっています。

11.2 html / body の基本（7〜10行目）

コピー

```
html, body {
  /* 画面の高さいっぱいを使えるように */
  height: 100%;
  /* 既定フォント。Arialが無ければゴシック系 */
  font-family: 'Arial', sans-serif;
}
```

▶ ここで初めて出てくるコード

- **`height:`**（CSS）：高さ。`height: 100%;` は親の高さいっぱいです。
- **`font-family:`**（CSS）：CSS で、文字の書体（フォント）を決める項目。`font-family: sans-serif;` は「飾りの無い書体」。
- **`A, B`**（CSS）：CSS の `button, .btn { … }` のように `,` で区切ると、「`button` と `.btn` の両方」に、同じ見た目を付けます。
- **`数%`**（CSS）：CSS の `100%` は、「入れ物（親）の大きさの何パーセントか」で表す単位です。

- `html, body` … カンマで2つの要素をまとめて指定。

- `height: 100%` … html と body の高さを「親（＝表示領域）の100%」に。これで後の `min-height: 100vh` などが効きやすくなります。
- `font-family` … 文字の書体。複数指定し、左から「使えれば使う」。`sans-serif`（サンセリフ）は「ゴシック体系のどれか」という最後の保険（特定のフォントが無くても、必ず何かのゴシック体になる）。

11.3 フォーム部品のリセット（12〜23行目）

> ▼ **このブロックがやること:** ボタン・入力欄・選択欄・複数行入力が持つ、ブラウザ既定の見た目（枠・背景・フォント）を消し、フォントを親から継承させて統一します。

コピー

```
button,
input,
select,
textarea {
  /* 親要素のフォントを引き継ぐ（部品だけ別フォントになるのを防ぐ） */
  font-family: inherit;
  /* 文字サイズも親基準に */
  font-size: 100%;
  margin: 0;
  padding: 0;
  /* 既定の枠線を消す（後で自分で付け直す） */
  border: none;
  /* 既定の背景を消す */
  background: none;
  /* クリック時の青い枠を消す（focusで自分で付ける） */
  outline: none;
}
```

> ▶ **ここで初めて出てくるコード**
>
> - **`border:`**（CSS）: CSS で、箱の枠線（太さ・線の種類・色）をまとめて決める項目。`border: 1px solid #e2e8f0;` 。
> - **`background:`**（CSS）: CSS で、背景（色や画像）を決める項目。`background: var(--color-bg);` は、色の変数の色にします。
> - **`outline:`**（CSS）: 部品の外側の線（入力の場所になった時に出る枠など）。`outline: none;` で消します。

用語: inherit（インヘリット）「親要素の値を引き継ぐ」キーワード。`<button>` や `<input>` は、何もしないと別フォント（ブラウザ既定）になりがちなので、ここで「親に合わせろ」と命じています。

⚠️ `outline: none` で、入力欄をクリックしたときの「青い枠」を消しています。ただし消しっぱなしだと「今どこを選択中か」が分からなくなるので、本アプリは `:focus` で自前の枠を付け直しています（§12.4）。これは見た目とアクセシビリティのバランスをとった設計です。

11.4 リスト・リンク・画像のリセット（25〜41行目）

コピー

```
ul,
ol {
  /* 箇条書きの「・」や番号を消す */
  list-style: none;
  margin: 0;
  padding: 0;
}

a {
  /* リンクの下線を消す */
  text-decoration: none;
  /* リンクの青色をやめ、親の文字色を継承 */
  color: inherit;
}

img {
  /* 画像が親より大きくならないように（はみ出し防止） */
  max-width: 100%;
  /* 縦横比を保ったまま縮小 */
  height: auto;
  /* 画像下の謎の隙間を消す（inline要素の癖を回避） */
  display: block;
}
```

▶ ここで初めて出てくるコード

- **`list-style:`**（CSS）: CSS で、並び（`ul`・`ol`）の点や番号の形を決める項目。`none` で消せます。
- **`text-decoration:`**（CSS）: CSS で、文字の下線などを決める項目。`none` で、リンクの下線を消せます。

- `ul, ol` … 箇条書きの記号（・や1.2.3.）を消し、ナビゲーションなどに使いやすくします。
- `a` … リンクの「下線」と「青文字」という既定スタイルを消し、必要な場所だけ後で付け直します（§11.7・§13）。

- `img` … `max-width: 100%; height: auto;` は **画像の鉄板**。親の幅に収まりつつ、縦横比が崩れません。`display: block` は、画像の下にできる謎の隙間（インライン要素の行ベースラインの癖）を消すおまじないです。本アプリのロゴ画像もこの恩恵を受けます。

11.5 CSS変数の定義（47〜51行目）

> **▼ このブロックがやること:** よく使う値（見出しサイズ）に名前を付けて、後でまとめて使い回せるようにします。これを **CSS変数（カスタムプロパティ）** と呼びます。

コピー

```
/* :root は html要素＝最上位。ここで定義すると全体で使える */
:root {
  /* 見出し1のサイズ（画面幅で伸縮、§6.2のclamp） */
  --heading1: clamp(1.5rem, 1.5rem + 3vw, 4rem);
  /* 見出し2のサイズ */
  --heading2: clamp(0.7rem, 0.8rem + 3vw, 1.5rem);
  /* 段落のサイズ */
  --paragraph: clamp(0.3rem, 0.6rem + 3vw, 1rem);
}
```

> **▶ ここで初めて出てくるコード**
>
> - **`:root`**（CSS）: CSS の `:root` は、ページ全体（いちばん外側の `<html>`）を選ぶ書き方です。ページのどこからでも使う色などの変数（`--color-text`）を、ここに書きます。
> - **`--名前`**（CSS）: CSS の `--color-text: #0f172a;` のように `--` で始まる名前は、自分で決める変数（カスタムプロパティ）です。`color: var(--color-text);` で使います。色を 1 か所で決めておけば、変える時も 1 か所で済みます。
> - **`clamp()`**（CSS）:「いちばん小さい値・ふつうの値・いちばん大きい値」を並べて、その間に収める書き方。`clamp(1.5rem, 1.5rem + 3vw, 4rem)` は「画面の幅に合わせて大きくなるが、1.5rem より小さく、4rem より大きくはならない」。
> - **`数rem`**（CSS）: CSS の `1.5rem` の `rem` は、「ページの基本の文字の大きさ（ふつう 16px）の何倍か」で表す単位です。利用者が文字を大きくする設定にすると、一緒に大きくなるので、読みやすさのために、文字や余白によく使います。
> - **`数vw`**（CSS）: `3vw` の `vw` は「画面の幅の 1%」です。`100vw` で画面の幅いっぱいになります。

- `--名前: 値;` で変数を **定義** し、`var(--名前)` で **呼び出し** ます（次の§11.6で使用）。
- `:root`（ルート）は文書の最上位（html）を指す特別なセレクタ。ここで定義すると全画面で使えます。
- 値を1か所で管理できるので、「見出しサイズを変えたい」ときは、ここだけ直せば全体に反映されます。

> **用語: CSS変数（カスタムプロパティ）** プログラミングの「変数（値に名前を付けて使い回す箱）」のCSS版。`--` で始まる名前を自分で決めて定義します。

11.6 body・見出し・段落（53〜81行目）

> **▼ このブロックがやること:** 本アプリ全画面の「土台レイアウト」をここで決めています。背景色・中央寄せ・最低の高さ・横スクロール、そして見出し/段落のサイズ（§11.5の変数を使用）。

コピー

```
body {
  font-family: 'Arial', sans-serif;
  margin: 0;
  /* 画面の縁に10pxの余白 */
  padding: 10px;
  /* 薄い灰色の背景 */
  background-color: #f4f7f9;
  /* 中身をFlexboxで配置 */
  display: flex;
  /* 横方向：中央寄せ（コンテンツを画面中央に） */
  justify-content: center;
  /* 縦方向：上寄せ */
  align-items: flex-start;
  /* 最低でも画面の高さ分は確保（vh=画面高さの1%） */
  min-height: 100vh;
  /* 横にはみ出たらスクロールバーを出す */
  overflow-x: auto;
  /* テキストを折り返さない（表が崩れないように） */
  white-space: nowrap;
}

h1 {
  /* §11.5で定義した変数を使用 */
  font-size: var(--heading1);
  /* 下に1文字分の余白 */
  margin-bottom: 1rem;
  /* 中央寄せ */
  text-align: center;
}

h2 {
  font-size: var(--heading2);
  margin-bottom: 0.2rem;
  text-align: center;
}

p {
  font-size: var(--paragraph);
  text-align: center;
}
```

▶ ここで初めて出てくるコード

- **`min-height:`**（CSS）：いちばん小さい時の高さ。`min-height: 100vh;` は「少なくとも画面の高さいっぱい」です。
- **`overflow-x:`**（CSS）：横にはみ出した時の扱い。`overflow-x: auto;` は「はみ出したら横のスクロールバーを出す」。
- **`white-space:`**（CSS）：CSS で、空白や改行をどう表示するかを決める項目。`pre-wrap` は「改行と空白はそのまま、はみ出す所は折り返す」。
- **`margin-bottom:`**（CSS）：CSS で、箱の下の外側の余白を決める項目。
- **`text-align:`**（CSS）：CSS で、文字の左右のそろえ方を決める項目。`center` で中央。
- **`数vh`**（CSS）：`100vh` の `vh` は「画面の高さの 1%」です。`100vh` で画面の高さいっぱいになります。
- **`var()`**（CSS）：CSS の `var(--color-text)` は、`--` で始まる変数の値を使う書き方です。

ここで本アプリの「全コンテンツが画面中央に配置される」基本レイアウトが決まっています。`body` 自体をFlexboxにして `display: flex; justify-content: center;` で中央寄せ——これが本アプリ全画面の土台です。`align-items: flex-start` で「縦は上寄せ」なので、コンテンツは上から並びます。

`min-height: 100vh` で「中身が少なくても画面の高さいっぱいは確保」、`overflow-x: auto` で「表などが横にはみ出たら横スクロールバーを出す」。`white-space: nowrap` は「文字を勝手に折り返さない」——表のセルが改行で崩れるのを防ぐ狙いです（§3⑩のログインボタンが折り返さないのもこの仲間の指定）。

11.7 リンクのホバー（83〜89行目）

コピー

```
a {
  /* リンクの下線を消す（§11.4で消したものの再指定） */
  text-decoration: none;
}

a:hover {
  /* ただしマウスを乗せた時だけ下線を出す */
  text-decoration: underline;
}
```

「普段は下線なし、ホバー時だけ下線」という、よくあるリンクの演出です。`text-decoration` は「文字の装飾（下線・取り消し線など）」。`none` =なし、`underline` =下線。

11.8 form（91〜109行目）

> **▼ このブロックがやること:** すべてのフォームを「白背景・角丸・影付き・中央寄せのカード」に見せます。さらに、狭い画面では幅の制約を緩めます。§3③の `<form>` がこのスタイルになります。

コピー

```
form {
  /* 白背景 */
  background-color: #fff;
  padding: 0;
  /* 角を丸く（半径8px） */
  border-radius: 8px;
  /* 影：右下方向に薄い影（立体感） */
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
  /* 最小幅350px */
  min-width: 350px;
  /* 最大幅450px */
  max-width: 450px;
  /* 上下10px、左右auto＝横中央寄せ */
  margin: 10px auto;
  display: flex;
  /* 中身を縦並びに */
  flex-direction: column;
  box-sizing: border-box;
}

/* 画面幅768px以下のとき */
@media (max-width: 768px) {
  form {
    width: auto;
    /* 最小幅の制約を外し、狭い画面に対応 */
    min-width: unset;
  }
}
```

> **▶ ここで初めて出てくるコード**
>
> - **`border-radius:`**（CSS）: CSS で、箱の角の丸みを決める項目。
> - **`box-shadow:`**（CSS）: 部品に影を付けます。`box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);` は「横 0・下へ 4px・ぼかし 6px の、薄い黒の影」。

- `border-radius: 8px` … 角を半径8pxで丸める。カードらしい柔らかい印象に。
- `box-shadow: 0 4px 6px rgba(0,0,0,0.1)` …「横0・下4px・ぼかし6px・黒の透明度10%」の影。カードを背景から浮き上がらせます。

- `min-width: 350px; max-width: 450px` … フォームの幅を350〜450pxの範囲に。広すぎず狭すぎず。
- `margin: 10px auto` … 左右autoでフォームが画面中央に来ます（§5.2）。
- メディアクエリで、768px以下では `min-width` の制約を外し、小さな画面で横にはみ出さないようにしています。

11.9 table（111〜129行目）

> **▼ このブロックがやること:** 表（一覧画面で多用）の罫線を1本にまとめ、セルに余白と中央寄せを付け、見出しセル（th）の白文字に黒い縁取りを付けて読みやすくします。

コピー

```
table {
  width: auto;
  /* セルの境界線を重ねて1本に（二重線を防ぐ） */
  border-collapse: collapse;
  margin-top: 10px;
  /* セル内で折り返さない */
  white-space: nowrap;
}

th,
td {
  /* 薄い灰色の枠線 */
  border: 1px solid #ddd;
  /* セル内の余白 */
  padding: 10px;
  /* 中央寄せ */
  text-align: center;
}

th {
  /* 見出しセルの文字は白 */
  color: white;
  font-weight: bold;
  text-shadow: -0.5px -0.5px 0 #555, 0.5px -0.5px 0 #555,
               /* 文字を4方向に縁取り（白文字を見やすく） */
               -0.5px 0.5px 0 #555, 0.5px 0.5px 0 #555;
}
```

▶ ここで初めて出てくるコード

- `border-collapse:`（CSS）: 表のマスの線の重なり方。`border-collapse: collapse;` で、となりのマスの線を 1 本にまとめます。
- `font-weight:`（CSS）: CSS で、文字の太さを決める項目。`700` や `bold` で太字、`800` はもっと太い字です。
- `text-shadow:`（CSS）: 文字に影を付けます。向きを変えた影をいくつも重ねると、文字のふち取りになります。

- `border-collapse: collapse` … 隣り合うセルの罫線を **1本にまとめる** 必須テク。これが無いと罫線が二重線になり、もたついて見えます。
- `th, td` … 見出しセルとデータセルの両方に、薄い枠線・余白10px・中央寄せ。
- `th` の `text-shadow` … 同じ影を **4方向**（左上・右上・左下・右下に各0.5px）に重ねることで、**白文字に黒い縁取り** を作っています。背景色（後述の `th-collar` など）の上でも白文字が読めるようにする工夫です。一覧画面（`KosuList` 等）の表ヘッダはこれです（§13）。

用語: text-shadow（テキストシャドウ） 文字に影を付けるプロパティ。横ずれ 縦ずれ ぼかし 色 の順。4方向に小さくずらした影を重ねると「縁取り（フチ文字）」になります。

11.10 ボタン群（131〜275行目）

本アプリには色違いのボタンが **7種類** あります。構造はすべて同じで、**背景色とホバー色だけ** が違います。代表として **青ボタン** を全解説し、他は差分の表で示します。

▼ このブロックがやること: `class="blue_button"` を付けた要素を、青背景・白文字（縁取り付き）・角丸・指カーソル・ホバーで濃い青になるボタンにします。§3⑩のログインボタンがこれです。

コピー

```
.blue_button {
  /* 青背景 */
  background-color: #007bff;
  /* 白文字 */
  color: #fff;
  /* 文字の縁取り（§11.9と同じ） */
  text-shadow: -0.5px -0.5px 0 #555, 0.5px -0.5px 0 #555, -0.5px 0.5px 0 #555, 0.5px 0.5px 0 #555;
  /* 内側余白＝ボタンの厚み */
  padding: 10px;
  font-size: 1rem;
  /* 枠線なし */
  border: none;
  /* 角丸 */
  border-radius: 4px;
  /* マウスを乗せると指マークに（押せる合図） */
  cursor: pointer;
  /* 色変化を0.3秒かけて（§9.1） */
  transition: background-color 0.3s ease;
  /* 文字を横書きに固定 */
  writing-mode: horizontal-tb;
  /* 文字を折り返さない */
  white-space: nowrap;
  width: auto;
  max-width: 100%;
}

.blue_button:hover {
  /* ホバーで濃い青へ */
  background-color: #0056b3;
  text-decoration: none;
}
```

▶ ここで初めて出てくるコード

- **`cursor:`**（CSS）: CSS で、マウスを乗せた時の矢印の形を決める項目。`pointer` は、押せる物の「指の形」です。
- **`writing-mode:`**（CSS）: 文字を書く向き。`horizontal-tb` は横書き（上から下へ行が進む）、`vertical-rl` は縦書きです。

- `cursor: pointer` … マウスを乗せると指マーク（👆）に変わり、「押せる」と分かります。
- `writing-mode: horizontal-tb` … 文字を必ず横書きに（縦書きになる事故を防ぐ保険）。
- `white-space: nowrap` … ボタンの文字を途中で折り返さない。
- `:hover` で背景色だけ濃い青に変え、`transition` のおかげで0.3秒かけてじわっと変化します。

他の6つのボタンは **背景色とホバー色だけ違い、構造は同一** です（黄・橙・桃・灰には左右marginも追加）：

クラス	通常色	ホバー色	主な用途
`blue_button`	`#007bff` 青	`#0056b3`	標準アクション（ログイン等）
`yellow_button`	`#ffd700` 黄	`#ffa500`	注意・編集
`green_button`	`#0f0` 緑	`#32cd32`	登録・実行
`light_blue_button`	`#0ff` 水	`#0af`	検索・絞り込み
`orange_button`	`#f50` 橙	`#f10`	強い操作（左右marginあり）
`pink_button`	`#ff8bc5` 桃	`#ff1493`	補助（左右marginあり）
`gray_button`	`#656565` 灰	`#3a3a3a`	戻る・キャンセル（左右marginあり）

> **保守のヒント:** 「あるボタンの色を変えたい」なら、そのボタンクラスの `background-color` と `:hover` の `background-color` の2か所を直せばOK。全画面のそのボタンに反映されます。逆に言うと、ここを直すと **全画面に影響する** ので、特定画面だけ変えたいときは `.module.css` 側で上書きします（§13・§14）。

11.11 アラート表示（277〜295行目）

> **▼ このブロックがやること:** `role="alert"` を持つ `<div>`（エラーメッセージ）を、赤系の警告ボックスにし、先頭に⚠️を自動で付けます。`Login.tsx` のエラー表示（§3⑨）がこれです。

なぞってコピーすると絵文字が別の文字になります。正しくは → コピー

```
/* role属性が alert の div を狙う（属性セレクタ） */
div[role="alert"] {
  /* 薄い赤背景 */
  background-color: #f8d7da;
  /* 濃い赤文字 */
  color: #721c24;
  /* 赤系の枠 */
  border: 1px solid #f5c6cb;
  border-radius: 4px;
  padding: 10px;
  /* 上0・左右auto(中央)・下16px */
  margin: 0 auto 16px auto;
  font-size: 0.9rem;
  /* 改行やスペースを保持して表示 */
  white-space: pre-wrap;
  /* 長い単語を折り返す */
  overflow-wrap: break-word;
  /* どこででも折り返してはみ出し防止 */
  word-break: break-all;
  /* 画面幅の90%まで */
  max-width: 90vw;
  box-sizing: border-box;
}

/* アラートの中身の「前」に */
div[role="alert"]::before {
  /* ⚠️ マークを自動で足す（§8.2の擬似要素） */
  content: "⚠️ ";
  font-size: 1.1rem;
}
```

▶ ここで初めて出てくるコード

- **`overflow-wrap:`**（CSS）: CSS で、長い単語（URL など）が箱からはみ出す時に、途中で折り返すかを決める項目。`anywhere` で、どこでも折り返せます。
- **`word-break:`**（CSS）: 長い言葉の折り返し方。`word-break: break-all;` は、英語の単語の途中でも折り返して、はみ出さないようにします。
- **`[属性]`**（CSS）: 属性（タグに書いた `type="date"` など）で選ぶ書き方（属性セレクタ）。`input[type="date"]` は日付の入力欄、`div[role="alert"]` は `role="alert"` の `<div>` を選びます。

- `div[role="alert"]` は **属性セレクタ**。「`role` 属性の値が `alert` の div」だけを狙います。`Login.tsx` がエラー時に出す `<div role="alert">`（§3⑨）に、自動でこのスタイルが付きます。

- `white-space: pre-wrap` … サーバーから来たエラーメッセージの改行をそのまま表示。
- `word-break: break-all` … 長いURLなどでも、どこでも折り返してはみ出しを防ぐ。
- `::before` で中身の **前** に「⚠️ 」を自動で足す。HTMLには書かないのに警告マークが出るのは、この擬似要素のおかげです。

表示例（ログインで未登録の従業員番号を入れた場合のイメージ）：

⚠ 登録されていない従業員番号です。

背景 #f8d7da（薄い赤）
文字 #721c24（濃い赤）

図3-3: エラーの表示の例。ログインで登録されていない従業員番号を入れた時に、薄い赤の背景・濃い赤の文字で出る

11.12 ナビゲーション（297〜311行目）

コピー

```
nav {
  display: flex;
  /* メニューリンクを中央に */
  justify-content: center;
  /* リンク間の隙間15px */
  gap: 15px;
  margin-bottom: 20px;
}

nav a {
  background: none;
  padding: 0;
  border-radius: 0;
  /* メニューのリンクは下線付き */
  text-decoration: underline;
  font-weight: normal;
  transition: color 0.3s ease;
}
```

> **▶ ここで初めて出てくるコード**
>
> - **`A B`（CSS）**：CSS の `.search-bar input` のように、空白で区切って並べると、「`.search-bar` **の中にある** `input` 」を選びます（子孫セレクター）。

- `nav` … セマンティックな「メニュー領域」タグ。Flexboxで中央寄せ、リンク間に15pxの隙間。

- `nav a` … メニュー内のリンク。`text-decoration: underline` で下線を付け（§11.4で全リンクの下線を消したので、ここで復活）、`transition: color 0.3s` で色変化をなめらかに。各一覧画面はこの `nav` の中で、画面ごとの色（§13の `kosu-nav` など）を上塗りします。

11.13 日付入力の調整（313〜342行目）

> **▼ このブロックがやること:** `<input type="date">`（日付入力欄）のブラウザ既定のごちゃごちゃしたUI（カレンダーアイコン・上下ボタン・クリアボタン）を整理し、「入力欄のどこをクリックしてもカレンダーが開く」使い勝手にします。

コピー

```
input[type=date]::-webkit-calendar-picker-indicator {
  /* カレンダーアイコンを重ねて配置 */
  position: absolute;
  /* 入力欄全体に広げ */
  width: 100%;
  height: 100%;
  /* 透明にして（見えないがクリック範囲は残る） */
  opacity: 0;
}

input[type="date"]::-webkit-inner-spin-button {
  /* 上下ボタンを消す */
  -webkit-appearance: none;
}

input[type="date"]::-webkit-clear-button {
  /* クリアボタンを消す */
  -webkit-appearance: none;
}

input[type="date"] {
  /* 上の絶対配置(absolute)の基準点になる */
  position: relative;
}

.MuiPickersInputBase-root .css-15ni0jc {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  opacity: 0;
  /* 重なり順を前面に */
  z-index: 1;
  cursor: pointer;
}
```

▶ ここで初めて出てくるコード

- **`position:`**（CSS）：置き方の決まり。`relative`（ふつうの位置から少しずらす）、`absolute`（近くの `relative` の親を基準に、好きな位置へ置く）、`fixed`（画面に固定して、スクロールしても動かない）などがあります。
- **`webkit-appearance:`**（CSS）：`-webkit-appearance: none;` は、ブラウザーが決めている入力欄などの見た目を消して、自分の CSS で見た目を作れるようにします（`-webkit-` は Chrome・Edge・Safari 向けの印）。
- **`top:`**（CSS）：`position` を決めた部品の、上からの位置。`top: 0;` で上の端に付きます。
- **`left:`**（CSS）：`position` を決めた部品の、左からの位置。
- **`z-index:`**（CSS）：重なった時の上下の順番。数が大きいほど上（手前）に出ます。`position` を決めた部品で効きます。
- **`::-webkit-calendar-picker-indicator`**（CSS）：日付の入力欄の右にある、カレンダーの印の部分（Chrome・Edge など）。ここに CSS を当てて、印を消したり、欄全体を押せるようにしたりします。
- **`::-webkit-inner-spin-button`**（CSS）：数や日付の入力欄の、上下の小さな矢印のボタンの部分（Chrome・Edge など）。`display: none;` で消せます。
- **`::-webkit-clear-button`**（CSS）：日付の入力欄の、中身を消す × ボタンの部分（古い Chrome など）。`display: none;` で消せます。

- `-webkit-` は **Chrome / Edge / Safari 向けの接頭辞**（ベンダープレフィックス）。これらのブラウザ独自の部品を狙うために必要です。
- カレンダーアイコンを `opacity: 0`（透明）で入力欄全体に広げることで、「どこをクリックしてもカレンダーが開く」を実現。
- `position: relative`（基準）と `position: absolute`（その基準に対して重ねる）はセットで使います。
- 末尾の `.MuiPickersInputBase-root .css-15ni0jc` は、Material-UI（本アプリが使うUI部品ライブラリ。第6章）の日付ピッカーに同じ細工をしたものです。本アプリの日付検索（`KosuList` の検索バー等）の使い勝手のための調整です。

用語: z-index（ゼットインデックス） 要素の「重なり順（前後）」。数字が大きいほど手前に来ます。`z-index: 1` で、透明なクリック領域を前面に置き、確実にクリックを拾います。

これで `global.css` 全342行を読み切りました。本アプリの「全画面共通の見た目」——中央寄せレイアウト・カード状フォーム・色違いボタン7種・表・アラート・日付入力の細工——のすべてがここにあります。

12. 実コード全解説：Login.module.css 全100行を1行ずつ

次に、ログイン画面 **専用** のスタイル `frontend/src/styles/MainPage/Login.module.css` を全部読みます。`global.css`（共通）との **使い分け** が体感できます。

12.1 出現アニメーション（1〜21行目）

> ▼ **このブロックがやること:** 「下からふわっと現れる」アニメーション（fadeInUp）を定義し、ログイン画面全体を囲む箱（login-wrapper）に適用します。§3①の一番外側の `<div>` がこれです。

コピー

```
/* 「下からふわっと」アニメの定義（§9.2） */
@keyframes fadeInUp {
  from {
    /* 開始：透明 */
    opacity: 0;
    /*        50px下にずらした位置から */
    transform: translateY(50px);
  }
  to {
    /* 終了：不透明 */
    opacity: 1;
    /*        定位置へ */
    transform: translateY(0);
  }
}

.login-wrapper {
  /* 画面全体を1秒かけて出現させる */
  animation: fadeInUp 1s ease-out;
  width: 100%;
  max-height: 100%;
  /* 画面幅を超えない */
  max-width: 100vw;
  /* 横はみ出しはスクロール */
  overflow-x: auto;
  overflow-y: auto;
  /* iOSでのスクロールを滑らかに */
  -webkit-overflow-scrolling: touch;
  /* 横中央寄せ */
  margin: 0 auto;
}
```

▶ ここで初めて出てくるコード

- `max-height:`（CSS）: いちばん大きい時の高さ。これより高くはなりません。
- `overflow-y:`（CSS）: 縦にはみ出した時の扱い。`overflow-y: auto;` は「はみ出したら縦のスクロールバーを出す」。
- `webkit-overflow-scrolling:`（CSS）: `-webkit-overflow-scrolling: touch;` は、iPhoneなどで、はみ出した部分を指でなめらかにスクロールできるようにする古い指定です。

- `animation: fadeInUp 1s ease-out` … 上で定義した `fadeInUp` を、1秒・`ease-out`（最後ゆっくり）で再生。ページを開くと、この箱ごと **下から1秒かけてふわっと** 現れます。
- `max-width: 100vw` … 画面幅を絶対に超えない（横スクロール暴発の防止）。
- `-webkit-overflow-scrolling: touch` … iPhone/iPad（iOS）で、指でのスクロールを慣性付きで滑らかにする指定。

12.2 入力エリア search-bar（23〜31行目）

▼ **このブロックがやること:** ラベル・入力欄・ボタンを「縦並び・中央寄せ・隙間15px」で配置します。§3⑤の `<div className={styles["search-bar"]}>` がこれです。

コピー

```
.search-bar {
  display: flex;
  /* 縦並び（ラベル→入力→ボタンを縦に） */
  flex-direction: column;
  justify-content: center;
  /* 中央寄せ */
  align-items: center;
  /* 各要素の間隔15px */
  gap: 15px;
  margin: 20px 0;
  flex-wrap: wrap;
}
```

⚠️ **同名クラスの方向違いに注意:** ここ（ログイン）の `search-bar` は `flex-direction: column`（縦並び）ですが、一覧画面 `KosuList` の `search-bar` は `row`（横並び）です（§13）。CSS Modules（§14）のおかげで名前が衝突せず、画面ごとに別の意味を持てます。

12.3 ロゴ login-logo（33〜40行目）

コピー

```
.login-logo {
  /* 最大400px だが画面が狭ければ90%まで（min関数） */
  max-width: min(400px, 90%);
  width: auto;
  /* 縦横比を保つ */
  height: auto;
  /* 横中央 */
  margin: 0 auto;
  /* 下に40pxの余白（入力エリアとの間隔） */
  margin-bottom: 40px;
  /* 画像を歪めず枠に収める */
  object-fit: contain;
}
```

> **▶ ここで初めて出てくるコード**
>
> - **`object-fit:`**（CSS）: 画像を枠に入れる時の合わせ方。`contain` は「縦横の比を保って、全部が見えるように収める」、`cover` は「枠いっぱいに広げて、はみ出しは切る」。
> - **`min()`**（CSS）: いくつかの値の中で、いちばん小さい物を使う書き方。`width: min(300px, 80%);` は「300px か 80% の、小さい方」。

- `min(400px, 90%)` … 「2つのうち小さい方」を採る（§6.2）。大画面では400px、小画面では画面幅の90%——どんな画面でもはみ出しません。
- `object-fit: contain` … 画像を **歪めずに** 枠に収める（縦横比キープ）。

12.4 入力欄とフォーカス（42〜54行目）

> **▼ このブロックがやること:** ログインの入力欄に余白・枠・角丸を付け、選択時（focus）に枠を青く光らせます。§11.3で `outline: none` した代わりの「選択中の合図」をここで作っています。

コピー

```
.search-bar input {
  /* 入力欄の内側余白（縦8px・横10px） */
  padding: 8px 10px;
  /* 灰色の枠 */
  border: 1px solid #999;
  border-radius: 4px;
  font-size: 1rem;
  /* 幅は「親の100%から60px引いた値」（calc関数） */
  width: calc(100% - 60px);
}

/* 入力欄が選択された時（§8.1の擬似クラス） */
.search-bar input:focus {
  /* 枠を青に */
  border-color: #08f;
  outline: none;
  /* 周囲にうっすら影＝光って見える */
  box-shadow: 0 0 3px #555;
}
```

▶ ここで初めて出てくるコード

- **`border-color:`**（CSS）: CSS で、枠線の色だけを決める項目。
- **`calc()`**（CSS）: 計算した値を使う書き方。`width: calc(100% - 40px);` は「親の幅から 40px 引いた幅」。`+` と `-` の前後には空白が要ります。

- `calc(100% - 60px)` … **計算式**（§6.2）。「親の幅いっぱいから60px分だけ狭く」。左右に余白を残します。
- `:focus` … 入力欄をクリック（選択）すると、枠が青（`#08f`）になり、`box-shadow` でうっすら光ります。「今ここに入力できますよ」の視覚的な合図です。

12.5 ヘルプボタン（菱形）（56〜100行目）

本アプリで最も凝ったスタイル。`Login.tsx` の `<Link className={styles["help-button"]}>`（§3⑪）を、**45度傾いた菱形に「?」が浮かぶボタン** にしています。

▼ このブロックがやること: 白い小さな四角を45度回して菱形にし、画面の右下に固定。**30秒後にふわっと出現** させる。中の「?」は文字だけ正位置に戻す。ホバーで青背景＋白文字に。

コピー

```
.help-button {
  display: flex;
  align-items: center;
  /* 中身（?）を中央に */
  justify-content: center;
  width: 20px;
  height: 20px;
  /* 白背景 */
  background-color: #ffffff;
  /* 青い枠 */
  border: 2px solid #007bff;
  /* 45度回転＝四角が菱形に見える */
  transform: rotate(45deg);
  /* 画面に固定（スクロールしても動かない） */
  position: fixed;
  /* 画面下から30px */
  bottom: 30px;
  /* 画面右から30px */
  right: 30px;
  text-decoration: none;
  cursor: pointer;
  /* 最初は透明（見えない） */
  opacity: 0;
  /* 30秒後に出現（後述） */
  animation: fadeIn 0s linear 30.0s forwards;
}

@keyframes fadeIn {
  to {
    /* 透明→不透明 */
    opacity: 1;
  }
}

/* ホバー時は色のみ変更（動きはなし） */
.help-button:hover {
  /* ホバーで青背景に（菱形が塗りつぶされる） */
  background-color: #007bff;
}

/* 菱形の中身として「?」を作る */
.help-button::after {
  content: "?";
  /* 菱形は45度傾いているので、文字だけ-45度で正位置に戻す */
  transform: rotate(-45deg);
  color: #007bff;
  font-weight: bold;
  font-size: 20px;
```

```
}

/* ホバー時の中の文字色 */
.help-button:hover::after {
  /* ホバー時、中の「?」を白に */
  color: #ffffff;
}
```

> **▶ ここで初めて出てくるコード**
>
> - **`bottom:`**（CSS）：`position` を決めた部品の、下からの位置。`bottom: 30px;` で下から 30px の所に置きます。
> - **`right:`**（CSS）：`position` を決めた部品の、右からの位置。
> - **`rotate()`**（CSS）：回す書き方。`rotate(45deg)` は右回りに 45 度回します。
> - **`数deg`**（CSS）：`45deg` の `deg` は角度の「度」（degree）です。

ここには高度なテクニックが詰まっています：

- **`transform: rotate(45deg)`** で正方形を菱形に見せ、中の `::after` の `?` だけ **`rotate(-45deg)`** で **文字の向きを元に戻す**。これをしないと「?」が斜めになってしまいます。「箱を傾けたら、中の文字も傾くので、文字だけ逆に傾け直す」という発想です。
- **`position: fixed; bottom: 30px; right: 30px;`** で **画面の右下に貼り付け**。`fixed` はスクロールしても動かない固定配置（§11.13の `absolute` と違い、基準は「画面そのもの」）。
- **`animation: fadeIn 0s linear 30.0s forwards`** … 形は「`fadeIn` を 0秒（一瞬）で、`linear` で、**30秒の遅延後** に再生、`forwards` 」。
- `0s` … アニメ自体の長さ（一瞬で透明→不透明）。
- `30.0s` … **遅延**（delay）。30秒待ってから始まる。
- `forwards` … アニメ終了後の状態（不透明）を **保ち続ける**。
- つまり「**ログイン画面を30秒見ていると、こっそり右下にヘルプが現れる**」仕掛けです。「すぐ出すと邪魔だが、迷っている人には助けを出したい」という配慮です。

> global と module の使い分けがここで腑に落ちる：`help-button`（この画面だけの凝った装飾）は module、`blue_button`（全画面共通のボタン）は global。「**共通は global、専用は module**」という設計が、実コードで実感できます。

これで `Login.module.css` 全100行を読み切りました。

---

13. 実コード追加解説：KosuList / MainMenu / Pagination

`global.css`（共通）と`Login.module.css`（専用）の関係が分かったところで、他の代表的な実CSSを見て、パターンの理解を固めます。

13.1 KosuList.module.css（一覧画面）— 表と横並び検索バー

工数一覧`frontend/src/styles/KosuPage/KosuList.module.css`から、特徴的な部分を読みます。

> ▼ **このブロックがやること:** 一覧画面全体を縦並び中央寄せにし、見出しやナビ・表ヘッダを「水色（#0ff）」のテーマカラーで統一します。

コピー

```
.kosu-list-wrapper {
  /* Loginと同じ出現アニメ（各ファイルにコピーされている） */
  animation: fadeInUp 1s ease-out;
  display: flex;
  /* 縦並び */
  flex-direction: column;
  justify-content: top;
  /* 中央寄せ */
  align-items: center;
  height: 100vh;
  width: 100%;
  margin: 0 auto;
  -webkit-overflow-scrolling: touch;
}

.th-collar {
  /* 表ヘッダの背景を水色に（global.cssのthの白文字＋縁取りが映える） */
  background-color: #0ff;
  text-shadow: -0.5px -0.5px 0 #555, 0.5px -0.5px 0 #555, -0.5px 0.5px 0 #555, 0.5px 0.5px 0 #555;
}

.status-ok {
  /* 「OK」状態を青太字で */
  color: blue;
  font-weight: bold;
}

.status-ng {
  /* 「NG」状態を赤太字で */
  color: red;
  font-weight: bold;
}
```

ポイント：

- `th-collar`は`global.css`の`th`（白文字＋黒縁取り。§11.9）に**背景色だけを足す** module クラス。`<th className={\`${styles["th-collar"]}\`}>`のように global と module を組み合わせて使います。「共通の土台＋画面ごとの色」という典型パターンです。

- `status-ok` / `status-ng` は、工数の入力状況を「青＝OK・赤＝NG」で色分け。一覧の表で「未入力の人」をひと目で分かるようにする工夫です。

横並びの検索バーとレスポンシブ分岐：

コピー

```
.search-bar {
  display: flex;
  /* 横並び（Loginのsearch-barはcolumn＝縦だった） */
  flex-direction: row;
  justify-content: center;
  align-items: center;
  gap: 15px;
  margin: 20px 0;
  /* 入りきらなければ折り返す */
  flex-wrap: wrap;
}

@media (max-width: 768px) {
  .search-bar {
    /* 狭い画面では縦並びに切り替え */
    flex-direction: column;
    gap: 10px;
  }
  .kosu-nav a {
    /* ナビ文字を小さく */
    font-size: 0.8rem;
  }
}
```

> **横並び→縦並びの切り替えが「レスポンシブ」の典型:** PCでは検索条件を横一列に並べ（`row`）、スマホ幅（768px以下）では縦積み（`column`）に切り替えます。同じ要素が画面幅で並び方を変える——これがメディアクエリ＋Flexboxの王道です。

13.2 MainMenu.module.css（メニュー画面）— リンクをボタンに見せる

メニュー画面 `frontend/src/MainPage/MainMenu.tsx` は、`<Link>`（本来はただのリンク）を **巨大なカラーボタン** に見せています。JSX側はこうです（§3で学んだ知識で読めます）：

コピー

```
{/* リンクだが見た目はボタン */}
<Link to="/kosu-menu" className={styles["kosu-menu-button"]}>
  {/* 左にアイコン */}
  <img src={Note} alt="工数アイコン" className={styles["icon"]} />
  {/* 中央にラベル */}
  工数MENU
</Link>
```

対応するCSS（水色の「工数MENU」ボタン）：

> ▼ **このブロックがやること:** `<Link>` を、横幅最大400px・水色背景・アイコンとラベルを両端に配置した大きなボタンに見せます。

コピー

```
.kosu-menu-button {
  width: 100%;
  max-width: 400px;
  /* 水色 */
  background-color: #0ff;
  color: #fff;
  /* 画面幅で文字サイズ伸縮（§6.2） */
  font-size: clamp(1rem, 1rem + 3vw, 2rem);
  border-radius: 4px;
  cursor: pointer;
  transition: background-color 0.3s ease, box-shadow 0.3s ease;
  display: flex;
  align-items: center;
  /* アイコンとラベルを両端に */
  justify-content: space-between;
  padding: 10px 15px;
  position: relative;
}
.kosu-menu-button:hover {
  /* ホバーで濃い水色 */
  background-color: #0af;
  /* 影で浮き上がる */
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
}
```

`MainMenu` には同じ構造で色違いのボタンが6つ（工数=水色・定義区分=緑・人員=黄・班員=橙・問い合わせ=桃・管理者=灰）並びます。`global.css` のボタン群（§11.10）と発想は同じ「色違いの量産」です。

擬似要素を使った「左右対称」の工夫と、スマホでの非表示：

コピー

```
.kosu-menu-button::after,
.def-menu-button::after,
/* ...他のボタンも... */ {
  /* 空の飾りを右端に置く */
  content: "";
  height: 1em;
  flex: 0 0 auto;
  /* 左のアイコンと同じ幅の「見えない箱」 */
  width: var(--icon-width, 1em);
}

@media (max-width: 768px) {
  .admin-menu-button {
    /* 狭い画面では管理者ボタンを隠す */
    display: none !important;
  }
}
```

▶ ここで初めて出てくるコード

- **`flex:`**（CSS）：`display: flex` の中の部品の伸び縮み。`flex: 0 0 auto;` は「伸びない・縮まない・大きさは中身に合わせる」。
- **`数em`**（CSS）：`1em` は「その部品の文字の大きさ 1 つ分」です。文字の大きさに合わせて伸び縮みします。
- **`!important`**（CSS）：「このスタイルをいちばん強くする」印。`display: none !important;` は、ほかの指定より必ず勝ちます。多く使うと、どれが効いているか分かりにくくなるので、必要な所だけに使います。

- 左にアイコン、右に同じ幅の **空の** `::after` を置くことで、`justify-content: space-between` でもラベルが **見た目の中央** に来るようにしています（左右のバランス取り）。
- `display: none !important` … スマホ幅では管理者メニューを **完全に隠す**（管理者操作はPC前提のため）。`!important` で確実に隠す数少ない正当な使用例です（§4.3）。

13.3 Pagination.module.css（共通部品）— 矢印型ボタン

ページ送り部品 `frontend/src/styles/Components/Pagination.module.css` は、`clip-path`（クリップパス）で **矢印型（三角）のボタン** を作っています。

▼ このブロックがやること: 「次へ」ボタンを、右向きの三角形（▶）に切り抜きます。`clip-path` は要素を任意の多角形に切り抜くプロパティです。

コピー

```
.pagination {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 15px;
  margin-top: 20px;
  white-space: nowrap;
}

.pagination .next-button {
  position: relative;
  width: 70px;
  height: 50px;
  /* 色はJS側から変数で渡す */
  background-color: var(--btn-bg-color, #fff);
  border: none;
  /* 左上→右中央→左下＝右向き三角 */
  clip-path: polygon(0% 0%, 100% 50%, 0% 100%);
  cursor: pointer;
  transition: background-color 0.3s ease, box-shadow 0.3s ease;
  display: flex;
  align-items: center;
  justify-content: flex-start;
  padding-left: 10px;
  color: #FFF;
  font-weight: bold;
  text-shadow: -0.5px -0.5px 0 #555, 0.5px -0.5px 0 #555, -0.5px 0.5px 0 #555, 0.5px 0.5px 0 #555;
}

.pagination button:disabled {
  /* 押せないときは灰色に */
  background-color: #999;
  color: #666;
  /* カーソルが「禁止マーク」に */
  cursor: not-allowed;
}
```

▶ ここで初めて出てくるコード

- **`clip-path:`**（CSS）：部品を好きな形に切り抜きます。`clip-path: polygon(0% 0%, 100% 50%, 0% 100%);` は 3 つの点を結んだ三角形（右向きの矢印の形）です。
- **`padding-left:`**（CSS）：左側の内側の余白。

- `clip-path: polygon(0% 0%, 100% 50%, 0% 100%)` … 3点（左上・右の中央・左下）を結んだ三角形に要素を切り抜く＝「▶」の形。「次へ」ボタンの矢印はこれで作っています。
- `var(--btn-bg-color, #fff)` … CSS変数を使い、色をJavaScript側から渡せるように。`, #fff` は「変数が未指定なら白」という既定値。
- `:disabled`（§8.1）… 最初のページで「前へ」が押せないとき、灰色＋禁止カーソルにして「押せない」と伝えます。`cursor: not-allowed` は禁止マーク（🚫）。

> **共通部品のCSSの考え方:** `Pagination` は全画面で使い回す「部品」なので、色は固定せず **CSS変数で外から差し替えられる** 設計になっています。これにより、工数一覧では水色、人員一覧では黄色…と、画面ごとに同じ部品を別色で使えます。

---

14. CSS Modulesのしくみ

§3・§12・§13で見た `styles["login-wrapper"]` の正体を説明します。

> **用語: CSS Modules（シーエスエス・モジュールズ）とは？** ファイルごとにCSSの効く範囲を **その画面だけに限定** するしくみ。ファイル名を `○○.module.css` にすると有効になります。

14.1 なぜ必要か

ふつうのCSS（`global.css` のような）は **全画面で共有** されます。すると、別々の画面で偶然同じクラス名（例 `.search-bar` ）を使うと **衝突** して、片方の見た目が壊れます。実際、本アプリには `Login` と `KosuList` の両方に `.search-bar` があり、しかも **片や縦並び・片や横並び** です（§12.2・§13.1）。これがもしふつうのCSSなら、後から読み込まれた方に上書きされて事故ります。

CSS Modules は、各クラス名をビルド（変換）時に **一意（重複しない）な名前へ自動改名** することで、これを防ぎます。

14.2 動作

コピー

```
// styles というオブジェクトに各クラスが入る
import styles from "../styles/MainPage/Login.module.css";
// ...
// styles["login-wrapper"] を適用
<div className={styles["login-wrapper"]}>
```

ビルド後、`login-wrapper` は例えば `Login_login-wrapper__a1B2c` のような **重複しない名前** に自動変換されます。`KosuList` の `search-bar` は `KosuList_search-bar__x9Y8z` に、`Login` の `search-bar` は `Login_search-bar__p3Q4r` に——別名になるので、絶対に衝突しません。

14.3 global との併用

1つの要素に、module クラスと global クラスを **両方** 付けられます。`Login.tsx` がまさにそうです。

コピー

```
// global（文字列で直接）
<button type="submit" className="blue_button">ログイン</button>
{/* module（styles経由） */}
<div className={styles["search-bar"]}>
```

- global（`global.css`）… 文字列で直接 `className="blue_button"`。**全画面共通**。改名されない。
- module（`○○.module.css`）… `className={styles["..."]}`。**その画面専用**。自動改名される。

	global	module
ファイル名	`global.css`	`○○.module.css`
書き方	`className="名前"`	`className={styles["名前"]}`
効く範囲	全画面	そのファイルだけ
名前衝突	する（注意）	しない（自動改名）
用途	共通ボタン・表・アラート	各画面固有のレイアウト・色

> ⚠️ **よくあるミス:** module のクラスを `className="login-wrapper"`（文字列）と書いてしまうと、改名後の本当の名前と一致せず **スタイルが効きません**。module は必ず `styles["login-wrapper"]` の形で書きます。逆に global を `styles["blue_button"]` と書いても、global.css は module ではないので効きません。

---

15. 開発者ツール（F12）実践

見た目の調整は、ブラウザの **開発者ツール** で「現物を見ながら」進めるのが最速・最安全です。

> **用語: 開発者ツール（デベロッパーツール）とは？** ブラウザに最初から入っている「ページの中身を調べる・その場で実験する」ための道具一式。コードを保存せずに見た目を試せるので、壊す心配がありません。

15.1 開き方

- 画面で **右クリック** →「**検証**」、または **F12** キー。

15.2 Elements（エレメンツ）タブ — HTML/CSSを調べる・試す

1. 左上の「矢印アイコン」を押し、調べたい部分（例：ログインボタン）をクリック。
2. 右側に、その要素に効いている **CSSルール** が一覧表示される。

3. 値をその場で書き換えると、画面が **即座に変化**（**保存はされない** ので、安心して実験できる。リロードで元に戻る）。
4. 効いているクラス名（例 `blue_button`）が分かったら、その名前で `global.css` や `.module.css` を **検索** して、本当の修正をする。

> **CSS Modules の改名を逆引きするコツ:** module のクラスは `Login_login-wrapper__a1B2c` のような改名後の名前で表示されます。`__` の前の `login-wrapper` の部分を見れば、元のファイル（Login）と元のクラス名が分かります。その「元の名前」で `.module.css` を検索すれば、修正箇所にたどり着けます。

15.3 Console（コンソール）タブ — エラーを見る

- JavaScriptのエラーが赤字で出ます。「画面が真っ白」「ボタンが効かない」等のとき、まずここを見ます。エラーの1行目をそのまま検索すると、原因が分かることが多いです。

15.4 Network（ネットワーク）タブ — 通信を見る

- APIの通信（リクエスト/レスポンス）が見られます。「保存したのにデータが更新されない」等の調査で、第7章（バックエンド）の知識と合わせて活用します。

> **保守の黄金フロー:** 「① 直したい部分を右クリック→検証 → ② クラス名を特定 → ③ そのクラスをコードで検索（global か module か見極め） → ④ 修正 → ⑤ ブラウザで確認」。この順番で進めれば、迷わず・壊さず直せます。

---

16. アクセシビリティ

> **用語: アクセシビリティ（accessibility：アクセシビリティ）とは？** 目の不自由な方や、マウスを使えない方など、**誰もが使えるようにする** 配慮のこと。「access（アクセス） + ability（できること）」＝「誰でも使える度合い」。

本アプリにも配慮が入っています。

- `<label htmlFor="numberInput">`（§3⑥） … 入力欄に「これは従業員番号の欄」という意味を結びつけ、読み上げソフトが「従業員番号、入力欄」と説明できます。ラベルをクリックで入力欄にカーソルが入る利点も。
- `<div role="alert">`（§3⑨・§11.11） … `role="alert"` を付けると、読み上げソフトが **エラーを自動で読み上げ** ます。視覚に頼らずエラーに気づけます。
- `alt="業務工数システムロゴ"`（§3②） … 画像が表示できないときや読み上げ時に、代替の説明になります。`alt=""`（空）にすると「装飾画像なので読み飛ばす」の意味になり、これも適切な使い分けです。

> ⚠️ **保守時の注意:** 画面を直すときも、**ラベルと入力欄の結びつき（`htmlFor` / `id`）** と **画像の `alt`** を消さないようにしましょう。`outline: none`（§11.3）で選択枠を消した場合は、`:focus` の代替スタイル（§12.4）を必ず用意してください。さもないとキーボードだけで操作する人が「今どこにいるか」分からなくなります。

---

17. よくある落とし穴とトラブルシューティング

手を動かすとき、また保守でつまずいたとき用の早見表です。

症状	原因	対処
スタイルが効かない	module を文字列で書いた／global を styles で書いた	`styles["名前"]`（module）と `"名前"`（global）の使い分けを確認（§14.3）
module のクラスがそもそも効かない	クラス名のスペルミス、`.module.css` でないファイル名	開発者ツールで改名後の名前が出ているか確認
幅がはみ出す・崩れる	`box-sizing` 未設定で padding が幅に加算	`box-sizing: border-box`（本アプリは全要素に設定済み・§11.1）
中央寄せできない	Flexの軸を勘違い（`row` / `column` で担当が逆）	`flex-direction` を確認し、`justify-content` / `align-items` の担当方向を見直す（§7.2）
表に二重線が出る	罫線が重なっている	`border-collapse: collapse`（§11.9）
ホバーが効かない	`:hover` の対象セレクタ違い	開発者ツールで実際のクラス名を確認（§15）
余白が思った通りにならない	margin と padding の取り違え	margin=外側、padding=内側（§5）
画像が縦長/横長に歪む	width と height を両方固定	`height: auto` で縦横比を保つ（§11.4）
`;` のつけ忘れで以降が壊れる	行末のセミコロン漏れ	各行の末尾 `;` を確認（§4.1）
入力欄の選択枠が消えて不便	`outline: none` のみで代替なし	`:focus` で `border-color` / `box-shadow` を付ける（§12.4）
変更したのに反映されない	ブラウザのキャッシュ	スーパーリロード（Ctrl+Shift+R）

---

18. 演習問題

手を動かすと定着します。**ローカル環境**（自分のPC内・壊しても安全）で試してください。実験が終わったら必ず元に戻すこと。

1. **色を変える（global）**：`global.css` の `.blue_button` の通常色を緑（`#0f0`）に変え、ログインボタンの色が変わることを確認。あわせて `MainMenu` のログアウトボタン（同じ `blue_button`）も変わることを確認（global は全画面に効く実感）。終わったら元に戻す。
2. **余白を変える（module）**：`Login.module.css` の `.login-logo` の `margin-bottom` を `40px` → `80px` にして、ロゴと入力欄の間隔が広がることを確認。
3. **アニメ時間を変える**：`.login-wrapper` の `animation: fadeInUp 1s` を `3s` にして、出現がゆっくりになることを確認。
4. **ヘルプボタンを早く出す**：`Login.module.css` の `.help-button` の `animation` の遅延 `30.0s` を `2s` にして、2秒でヘルプが出ることを確認（実験後は元に戻す）。
5. **開発者ツール**：F12でログインボタンを検証し、`background-color` をその場で赤に変えてみる（リロードで元に戻ることも確認）。
6. **軸の入れ替えを体感**：`Login.module.css` の `.search-bar` の `flex-direction: column` を `row` に変え、ラベル・入力欄・ボタンが横並びになることを確認（§7.2の軸の話を実感）。
7. **レスポンシブを観察**：F12で画面幅を狭く（768px以下に）して、`KosuList` の検索バーが横並びから縦並びに変わること、`MainMenu` の管理者ボタンが消えることを確認（§13）。
8. **属性セレクタを確認**：開発者ツールで `<div role="alert">` を探し、`div[role="alert"]` と `::before` のルールが効いて「⚠」が出ていることを確認（§11.11）。

---

19. この章のまとめ

- 画面は **HTML（骨組み）** ＋ **CSS（見た目）** ＋ **JS（動き）** の3層。ブラウザはサーバーからHTMLを受け取り、**DOM**（木構造）を作り、CSSを当てて **レンダリング** する。JSがDOMを書き換えると再レンダリングされる。
- HTMLは **タグ** で構造を作り、**属性** で情報を足す。入れ子は交差させない。空要素はJSXでは `/>` で閉じる。本アプリは **フォーム**（form/label/input/button）が中心。JSXでは `class` → `className`、`for` → `htmlFor`、`{ }` でJSの値を埋め込む。
- CSSは **セレクタ** → **プロパティ:値;**。**カスケード**（優先順位：`#id` > `.class` >タグ、同強さは後勝ち、`!important` は最強だが乱用禁物）。
- 要点プロパティ群：**ボックスモデル**（margin/border/padding/content）と `box-sizing: border-box`、**Flexbox**（`display:flex` ＋ `flex-direction` ＋ `justify-content` / `align-items`、軸の取り違えに注意）、単位（px/rem/%/vw、`clamp` / `calc` / `min`）、色（16進・rgba）、擬似クラス（`:hover` / `:focus` / `:disabled`）・擬似要素（`::after` / `::before` ＋ `content`）、トランジション・`@keyframes` アニメ、メディアクエリ。

- 本アプリの `global.css` **全342行**（全要素リセット・CSS変数・中央寄せレイアウト・カード状フォーム・表＋縁取り白文字・色違いボタン7種・`role="alert"` ＋ ⚠️・日付入力の細工）と `Login.module.css` **全100行**（出現アニメ `fadeInUp`・入力フォーカス・30秒後に出る菱形ヘルプボタン）を1行ずつ読み切った。さらに `KosuList`（横並び検索バー＋レスポンシブ＋テーマ色）・`MainMenu`（リンクを巨大カラーボタンに）・`Pagination`（`clip-path` で矢印ボタン）の実例も確認した。
- **global＝全画面共通、module＝その画面専用。** `CSS Modules` がクラス名を自動改名して名前衝突を防ぐ。1要素に両方を併用できる（例: `<button className="blue_button">` ＋ `<div className={styles["search-bar"]}>`）。
- 見た目の調整・調査は **開発者ツール**（F12）→ **クラス名特定** → **コード検索**（global か module か見極め）→ **修正** → **確認** の流れで。アクセシビリティ（`htmlFor` / `id`・`role="alert"`・`alt`）は消さない。

次は、画面に動きを与える言語、「04_JavaScript_TypeScript.md」へ進みます。ここで何度も出てきた `onChange`・`useState`・`Number()`・`{条件 && 表示}` といった「動きの部分」の正体が、いよいよ明らかになります。